REWIND 入門

巻き戻せるプログラミングを、はじめから順に学ぶ本

目次

第1章 この本について

- REWIND とはどんな言語か
- この本の対象と読み方
- コード例の約束

第2章 準備と最初のプログラム

- 処理系を用意する
- Hello, REWIND
- 実行のしかた
- エラーの読み方

第3章 値・変数・基本の型

- 変数を作る
- 基本の型
- 暗黙の型変換はない
- 整数の計算は検査される
- 文字列
- 比較と等価
- 演算子の優先順位
- 型に別名を付ける

第4章 分岐と繰り返し

- if と else
- 式としての if
- while
- for
- match
- if let と let … else

第5章 関数

- 関数を宣言する
- effects の宣言
- 名前付きの引数と既定値
- クロージャ
- 関数を引数に渡す
- 再帰の深さには上限がある

第6章 List・Map・イテレータ

- List

Map
変更できる値には持ち主がいる
イテレータ
タプル

第7章 自分の型を定義する

record: 変わらないデータ
struct: 変えられるデータ
enum: 選択肢を表す
match で網羅を検査する
型にメソッドを足したいとき
公開と非公開

第8章 trait とジェネリクス

trait: 型が持つ約束
ジェネリクス
型引数に制約を付ける
自作の型を比べる
型引数付きの enum

第9章 所有権と借用

なぜ所有権があるのか
二種類の値
move: 持ち主を替える
借用: 貸すだけで、持ち主は変わらない
freeze と thaw: 変えない複製を共有する
使い分けの目安

第10章 エラーを扱う

二種類の失敗
Option: あるか、ないか
Result: 成功か、失敗か
? で失敗を呼び出し元へ返す
panic と assert
テストを書く

第11章 チェックポイント: commit と revert

保存して、戻る
何が戻るのか
revert は、実行位置を戻さない
resume の注意: 終わらないループ
保存点の名前と寿命
考え方のまとめ

第12章 公開と外部領域

三つの層
publish: 外へ確定する
main が戻ったときの自動公開
begin: 最初の状態へ戻る
branch: 試すだけ試す

外部領域: 取り直せない値を扱う

第13章 効果と許可

効果とは何か
効果の一覧
効果は呼び出し元へ伝わる
宣言と許可は別
ファイルのパス
Secret: 値を伏せる

第14章 module とプロジェクト

import
公開する名前を決める
module の中の状態
プロジェクトにする

第15章 並行処理

async fn と Task
決定的な協調スケジューラ
タスクへ渡せる値
失敗・取り消し・時間切れ
チャネル
TaskGroup: 片付け忘れを防ぐ

第16章 後始末: defer と using

defer
using
巻き戻しと後始末

第17章 入力・引数・ファイル

標準入力から読む
大量の数値を読む
起動引数
ファイルの読み書き
複数の書き込みは一括では確定しない
時刻と乱数
環境変数

第18章 文字列・Unicode・正規表現

「一文字」の数え方は一つではない
書記素と正規化
正規表現

第19章 JSON と CSV

JSON を読む
大きな JSON を逐次的に読む
CSV を読む
正確な値を保存する

第20章 大きな整数・十進数・日時

失敗を取り出す補助関数

BigInt: 桁数に上限のない整数
Decimal: 十進数で正確に計算する
日時

第21章 数値配列と線形代数

List と数値配列
連立方程式を解く
計算が重いとき

第22章 テスト・診断・記録と再生

テストを書く
エラーの診断を読む
実行を記録して、再生する
仕事量を調べる
エディタとデバッガ

第23章 実行予算と性能

予算がある理由
予算の種類
既定の予算は小さい
予算は巻き戻らない
ソースの中で指定する
呼び出しの深さと、そのほかの制限
重いと感じたら

第24章 外の世界とつなぐ: データベース・通信・GUI

共通の約束
データベース
HTTP の通信
ネイティブ GUI

第25章 実習: 在庫を「全部反映するか、一つも反映しないか」で更新する

設計を考える
プログラム
読み解きのポイント
発展: 取り組んでみよう

付録A 標準ライブラリの地図

付録B よくあるエラーと対処

付録C 用語集

付録D 現状の制約

付録E 次に読むもの

第1章 この本について

REWIND とはどんな言語か

プログラムを書いていて、「ここまでの状態に戻れたらいいのに」と思ったことはないでしょうか。探索で行き止まりに入ったとき、試しに変更した結果が気に入らなかったとき、あるいは途中で失敗したので最初からやり直したいとき。普通の言語では、そのための保存と復元を自分で書くことになります。

REWIND は、この「戻る」操作を言語の機能として持っているプログラミング言語です。変数や `List` の中身といった計算の状態に **名前を付けて保存** しておき、あとから **その状態へ戻れます**。まだ画面やファイルに出ていない出力も、一緒に取り消されます。

REWIND

```
fn main()->Int {
    var score = 10;
    commit before;        // いまの状態に before という名前を付けて保存する
    score = 99;
    Out.println(score);   // まだ外には出ていない出力（保留中）
    revert before;        // 保存した状態へ戻る。score は 10、99 の出力も消える
    Out.println(score);
    return 0;
}
```

出力

```
10
```

`99` は画面に出ません。`revert` が、保存したあとの変更と保留中の出力をまとめて取り消したからです。

ただし、何でも取り消せるわけではありません。すでに外へ出てしまったもの（表示した文字、送信済みの通信など）は、戻せません。REWIND は「戻せるもの」と「戻せないもの」の境目を言語の中にはっきり持っていて、その境目を理解することが、この言語を使いこなす中心になります。この本では、その境目を何度も例で確かめます。

この本の対象と読み方

対象は **REWIND 3.6.0** です。ほかの言語でプログラムを書いた経験があれば読み進められるように書いています。

- 第2章〜第8章は、変数・制御構造・関数・コレクション・型といった基本です。どの言語にも似たものがあるので、早く進めてかまいません。
- 第9章〜第14章が、REWIND らしい部分です。所有権、エラーの扱い、チェックポイント、公開と外部領域、効果、module を扱います。腰を据えて読んでください。

- 第15章以降は、並行処理、入出力、データ形式、数値、テスト、実行予算、外部とのやりとりといった、実用のための章です。必要なところから読めます。第25章は、全体をまとめる実習です。

言語仕様の正確な定義が必要なときは、別の文書 language-reference.md を引いてください。この本は理解しやすさを優先し、細かい規則や例外は省いています。標準ライブラリの全関数は libraries/README.md にあります。

コード例の約束

この本のコード例は、断りがなければ **そのまま動く完全なプログラム** です。`main.rw` という名前で保存し、`rewind run main.rw` で実行できます。例の直後の `text` ブロックは、その標準出力です。

エラーになる例には、見出しに「エラーになる例」と書き、起きるエラーの名前を添えています。

この本の例はすべて、出版前に実際の処理系で実行し、出力とエラーの種類を確かめてあります。本文と実際の動作がずれていたら、それは本の誤りですので、報告してください。

第2章 準備と最初のプログラム

処理系を用意する

REWIND の処理系は `rewind` という一つの実行ファイルです。構文の検査、実行、テスト、整形といった作業は、すべてこのコマンドで行います。

GitHub のリリースページから、お使いの OS 用の SDK（Linux x86_64 または Windows x64）を取得して展開し、`bin` ディレクトリを `PATH` に追加してください。署名とチェックサムの確認手順は getting-started.md にあります。次のコマンドで版が表示されれば準備は終わりです。

コマンド

```
rewind --version
```

ソースコードからビルドすることもできます。その場合は、リポジトリで `cargo build --release --locked` を実行し、できた実行ファイルを使います。

Hello, REWIND

空のフォルダーに `main.rw` というファイルを作り、次の内容を UTF-8 で保存します。

REWIND

```
fn main()->Int {
    Out.println("Hello, REWIND!");
    return 0;
}
```

出力

```
Hello, REWIND!
```

実行します。

コマンド

```
rewind run main.rw
```

この短いプログラムに、REWIND の基本がいくつか入っています。

- ソースファイルに書けるのは **宣言だけ** です。関数、型、定数などを宣言し、実行される文は関数の中に書きます。
- 実行は `fn main` から始まります。`main` は `Int` を返し、`0` は正常終了を表します。

- `Out.println` は、標準出力へ 1 行出力します。ただし、この出力はすぐには画面に出ません。いったん **保留中** になり、`main` が戻るときにまとめて外へ出ます。第12章で詳しく説明します。

実行のしかた

`rewind run` のほかに、よく使うコマンドを挙げておきます。

やりたいこと	コマンド
実行せずに、型や所有権の誤りだけ調べる	`rewind check main.rw`
ソースの形を整える	`rewind fmt main.rw`
実行用のファイル（`main.rwc`）を作る	`rewind compile main.rw`
作ったファイルを実行する（ソースは不要）	`rewind run main.rwc`
使い方を見る	`rewind --help`

プログラムに引数を渡すときは、`--` の後ろに書きます。たとえば `rewind run main.rw -- a b` です。

エラーの読み方

間違いがあると、`rewind` は実行する前に止まり、場所と原因を表示します。わざと間違えてみましょう。

エラーになる例

```
fn main()->Int {
    let x: Int = 3;
    Out.println(x);
    return 0;
}

fn helper() -> Unit effects {} {
    Out.println("hi");
}
```

`helper` は「外部とのやりとりをしない」と宣言（`effects {}`）しているのに、出力しています。REWIND はこれを実行前に見つけて拒否します。エラーの表示は次のような形です。

出力

```
error[MissingEffect]: helper: undeclared effect output
  --> main.rw:7:1
  help: Two separate steps: add the effect to the function's effects clause (a source change), and
let the person running the program grant it (--allow-effects or the manifest effects). A declarati
on alone grants nothing.
caused by[MissingEffect]: operation requires effect output
  --> main.rw:8:5
```

エラー名（`MissingEffect`）、場所（ファイル名:行:列）、対処のヒント（`help`）が並び、原因となった操作が `caused by` に続きます。ここでは、8 行目の出力が原因だと分かります。この本では、エラーを「怖いもの」ではなく、間違いを早く教えてくれる仕組みとして扱います。`effects` の意味は第13章で説明するので、いまは「宣言と実際が食い違っていた」と読めれば十分です。

第3章 値・変数・基本の型

変数を作る

名前に値を結び付けることを **束縛** と呼びます。`let` で作った名前は、あとから別の値に付け替えられません。付け替えたいときは `var` を使います。

REWIND

```
fn main()->Int {
    let base = 10;          // 付け替えられない
    var score = base;       // 付け替えられる
    score += 5;
    score *= 2;
    Out.println(score);
    return 0;
}
```

出力

```
30
```

迷ったら `let` を使い、必要になったときだけ `var` に変えるのがお勧めです。「この名前はずっと同じ値だ」と分かると、プログラムが読みやすくなります。

`let` の名前に再び代入しようとすると、実行前にエラーになります。

エラーになる例

```
fn main()->Int {
    let x = 1;
    x = 2;
    return 0;
}
```

型は、値から自動的に決まります。書きたいときは `名前: 型` で明示できます。

REWIND

```
fn main()->Int {
    let count: Int = 3;
    let ratio: Float = 0.5;
    Out.println(count);
    Out.println(ratio);
    return 0;
}
```

出力

```
3
0.5
```

基本の型

型	意味	例
`Int`	符号付き 64 ビット整数	`42` 、`-7` 、`1_000_000` 、`0xff`
`Float`	64 ビットの浮動小数点数	`3.14` 、`1.5e3`
`Bool`	真偽	`true` 、`false`
`String`	UTF-8 の文字列	`"こんにちは"`
`Bytes`	バイト列	`Bytes("abc")`
`Unit`	「値が無い」ことを表す型	`()`

数値のリテラルでは、桁の区切りに `_` を入れたり、`0x`（16 進）、`0b`（2 進）、`0o`（8 進）を使ったりできます。小数点の前後には数字が必要で、`.5` や `5.` とは書けません。

暗黙の型変換はない

REWIND には、型を自動で変換する仕組みがありません。`Int` と `Float` を混ぜた計算は、エラーになります。変換は、目的を書いて明示します。

REWIND

```
fn main()->Int {
    let n: Int = 3;
    let x: Float = 1.5;
    // n + x はエラー。まず Float に変換する
    match n.toFloatChecked() {
        Ok(f) => { Out.println(f + x); },
        Err(_) => { Out.println("変換できません"); }
    }
    Out.println(x.toIntChecked());   // 1.5 は整数にできないので失敗する
    return 0;
}
```

出力

```
4.5
Err(InexactOrOverflow)
```

`toFloatChecked` や `toIntChecked` は、変換が正確にできたときだけ `Ok` を返します。失敗の可能性を `Result` という型で表すのは、REWIND の基本的な方針です。`Ok` と `Err` 、`match` の使い方は、第10章で詳しく説明します。

整数の計算は検査される

`Int` の `+ - * / %` は、結果が 64 ビットの範囲を超えたり、0 で割ったりすると、**実行が止まります**。値が黙って丸められたり、折り返したりはしません。

エラーになる例

```
fn main()->Int {
    let max = 9223372036854775807;
    Out.println(max + 1);
    return 0;
}
```

除算は 0 に向かって切り捨て、剰余の符号は割られる数に合わせます。

REWIND

```
fn main()->Int {
    Out.println(7 / 2);
    Out.println(-7 / 2);
    Out.println(-7 % 3);
    Out.println(7 % -3);
    return 0;
}
```

出力

```
3
-3
-1
1
```

より大きな整数が必要なときは `std.bigint` を使います（第20章）。

`Float` は IEEE 754 の規則に従います。0 で割っても止まらず、無限大や NaN（数でない値）になります。

REWIND

```
fn main()->Int {
    Out.println(1.0 / 0.0);
    let nan = 0.0 / 0.0;
    Out.println(nan == nan);    // NaN はそれ自身とも等しくない
    Out.println(0.1 + 0.2);
    return 0;
}
```

出力

```
inf
false
0.30000000000000004
```

文字列

文字列は `"` で囲みます。改行や `\n` などのエスケープが使えます。文字列どうしは `+` で連結できます。

文字列の中に値を埋め込むには、`${式}` を使います。`${` という文字そのものを書きたいときは `$${` とします。

REWIND

```
fn main()->Int {
    let name = "REWIND";
    let n = 3;
    Out.println("${name}: ${n + 1} 件");
    Out.println("価格は $${n}");
    var s = "a";
    s += "b" + "c";
    Out.println(s);
    return 0;
}
```

出力

```
REWIND: 4 件
価格は ${n}
abc
```

数値などを文字列にしたいときは `.format()` も使えます（`3.format()` は `"3"` です）。

比較と等価

`<`、`<=`、`>`、`>=` は `Int`、`Float`、`String` に使えます。文字列は、UTF-8 のバイト列としての辞書順で比べます。論理演算は `&&`、`||`、`!` で、`&&` と `||` は左の結果で決まれば右を評価しません。

`==` は、値によって意味が変わります。数値、文字列、タプル、後の章で学ぶ `record` や `enum` は、**中身が等しいか** を調べます。`List` や `Map` のように変更できる値は、**同じ一つの値であるか** を調べるので、中身が同じでも別々に作ったものは等しくなりません。中身を比べたいときは `freeze` します（第9章）。

REWIND

```
fn main()->Int {
    let a = [1, 2];
    let b = [1, 2];
    Out.println(a == b);                    // 別々の List
    Out.println(freeze(a) == freeze(b));    // 中身で比べる
    Out.println((1, "x") == (1, "x"));
    Out.println(Some(3) == Some(3));
    return 0;
}
```

出力

```
false
true
true
true
```

演算子の優先順位

数学と同じように、`*`、`/`、`%` は `+`、`-` より先に計算されます。比較は算術よりも後、`&&` は `||` よりも先です。迷うときは `( )` で順序を明示してください。

ビット演算子（`&`、`|`、`^`、`<<`、`>>`）はありません。ビット操作が必要なときは、標準ライブラリの `std.bits` を使います。

型に別名を付ける

`type` で、既存の型に別名を付けられます。別名は新しい型を作らないので、元の型と自由に代入し合えます。意味を読み手に伝えるのに便利です。

REWIND

```
type Score = Int;

fn main()->Int {
    let s: Score = 3;
    Out.println(s + 1);
    return 0;
}
```

出力

```
4
```

第4章 分岐と繰り返し

if と else

条件は `Bool` でなければなりません。数値や文字列を条件に使うことはできません。

REWIND

```
fn grade(score: Int) -> String effects {} {
    if score >= 80 { return "A"; }
    else if score >= 60 { return "B"; }
    else { return "C"; }
}

fn main()->Int {
    Out.println(grade(90));
    Out.println(grade(65));
    Out.println(grade(10));
    return 0;
}
```

出力

```
A
B
C
```

この例の `effects {}` は、関数の宣言に付ける「外部とのやりとりをしない」という印です。第5章と第13章で説明するので、いまは「関数にはこう書くことが多い」と思っておいてください。

式としての if

REWIND では、`if` も `{ }` のブロックも **値を返せます**。ブロックは、最後の式（末尾に `;` を付けないもの）が値になります。分岐の値の型は、すべてそろっている必要があります。

REWIND

```
fn main()->Int {
    let n = 5;
    let size = if n > 3 { "big" } else { "small" };
    let doubled = { let t = n * 2; t + 1 };
    Out.println(size);
    Out.println(doubled);
    return 0;
}
```

出力

```
big
11
```

型がそろっていない分岐は、実行前にエラーになります。

エラーになる例

```
fn main()->Int {
    let x = if true { 1 } else { "a" };
    return 0;
}
```

while

REWIND

```
fn main()->Int {
    var i = 0;
    var total = 0;
    while i < 5 {
        i += 1;
        if i == 2 { continue; }   // 次の反復へ
        if i == 5 { break; }      // ループを抜ける
        total += i;
    }
    Out.println(total);
    return 0;
}
```

出力

```
8
```

`i` が 1、3、4 のときだけ足されるので、合計は 8 です。`break` と `continue` は、いちばん内側のループに働きます。

for

`for` には二つの使い方があります。

一つは **範囲** です。`a..b` は、`a` 以上 `b` **未満** の整数を表します。`b` は含まれません。`a` が `b` 以上なら、一度も実行されません。

もう一つは、`List` などの **コレクション** の要素を順に取り出す使い方です。`Map` は `keys()` でキーを取り出します。キーは常に昇順です。

REWIND

```
fn main()->Int {
    for i in 0..3 { Out.println(i); }
    for name in ["a", "b"] { Out.println(name); }

    let ages = Map<String, Int>();
    ages.set("bob", 28);
    ages.set("ann", 31);
    for key in ages.keys() { Out.println(key); }
    return 0;
}
```

出力

```
0
1
2
a
b
ann
bob
```

match

`match` は、値の形によって処理を分けます。他の言語の `switch` に似ていますが、値の中身を取り出せる点と、**すべての場合を扱っているかを検査してくれる** 点が違います。

REWIND

```
fn main()->Int {
    let n = 7;
    match n {
        0..5 => { Out.println("低"); },
        5..10 => { Out.println("中"); },
        _ => { Out.println("高"); }
    }
    let word = match n { 1 => "one", 2 => "two", _ => "many" };
    Out.println(word);
    return 0;
}
```

出力

```
中
many
```

整数や文字列の `match` では、残りのすべてを受ける `_` が必要です。`match` は式なので、上の `word` のように値を返せます。

値の中身を取り出すパターンは、第7章で `enum` と一緒に学びます。

if let と let … else

ある形のときだけ処理したい場合は、`match` より `if let` が簡潔です。

REWIND

```
fn main()->Int {
    let found: Option<Int> = Some(3);
    if let Some(v) = found {
        Out.println(v);
    } else {
        Out.println("なし");
    }
    return 0;
}
```

出力

```
3
```

`Option<Int>` は「`Int` がある（`Some`）か、ない（`None`）か」を表す型です。詳しくは第10章で扱います。

形が合わなければ関数から抜けたい、という場面には `let … else` が便利です。`else` の中は、`return`、`break`、`continue`、`panic` のどれかで終わらなければなりません。一致したときは、取り出した名前が後ろの文で使えます。

REWIND

```
fn firstOrZero(m: &Map<String, Int>) -> Int effects {} {
    let Some(v) = m.get("first") else { return 0; };
    return v;
}

fn main()->Int {
    let m = Map<String, Int>();
    Out.println(firstOrZero(&m));
    m.set("first", 9);
    Out.println(firstOrZero(&m));
    return 0;
}
```

出力

```
0
9
```

引数の `&Map<String, Int>` は「読むだけで借りる」という意味です。第9章で説明します。

第5章 関数

関数を宣言する

関数には、名前、引数と型、戻り値の型を書きます。戻り値は `return` で返します。

REWIND

```
fn factorial(n: Int) -> Int effects {} {
    if n <= 1 { return 1; }
    return n * factorial(n - 1);
}

fn main()->Int {
    Out.println(factorial(5));
    return 0;
}
```

出力

```
120
```

関数は、ファイルのどこに書いても、どこからでも呼べます。呼ぶ側より後ろに宣言してもかまいません。関数の中に別の関数を宣言することはできません。

値を返す関数は、すべての経路で `return` に到達する必要があります。足りないと実行前に拒否されます。

本体が一つの式だけの関数は、`=` で短く書けます。

REWIND

```
fn double(x: Int) -> Int = x * 2;
fn greet(name: String) -> String = "hi ${name}";

fn main()->Int {
    Out.println(double(21));
    Out.println(greet("REWIND"));
    return 0;
}
```

出力

```
42
hi REWIND
```

effects の宣言

関数のシグネチャに付いている `effects {}` は、その関数が **外の世界に対して何をするか** の宣言です。何もしない関数は `effects {}` と書きます。たとえば出力する関数は `effects {output}` と書きます。

REWIND

```
fn greet(name: String) -> Unit effects {output} {
    Out.println("こんにちは、${name}さん");
}

fn main()->Int effects {output} {
    greet("REWIND");
    return 0;
}
```

出力

```
こんにちは、REWINDさん
```

なぜこんな宣言が要るのでしょうか。理由は、巻き戻しにあります。「外部とやりとりしない関数」は、同じ引数なら同じ結果になるので、巻き戻して実行し直しても安全です。この区別を言語が追跡してくれるので、あとの章で出てくる「戻せるもの」と「戻せないもの」の境目を、型検査で守れます。詳しくは第13章で説明します。

なお、`main` だけは `effects` を省略でき、省略すると本体が必要とする効果が自動で求められます。

名前付きの引数と既定値

引数には、名前を付けて渡せます。名前を付けると、順序は自由です。引数には **既定値**（リテラルか `None`）を付けられ、その引数は呼ぶときに省略できます。

REWIND

```
fn box(width: Int, height: Int, label: String = "box") -> String effects {} {
    return "${label}: ${width * height}";
}

fn main()->Int {
    Out.println(box(3, 4));
    Out.println(box(height: 2, width: 5, label: "wide"));
    return 0;
}
```

出力

```
box: 12
wide: 10
```

既定値のある引数の後ろには、既定値のない引数を置けません。

クロージャ

名前の無い関数は、`|引数: 型| -> 戻り値の型 { 本体 }` で作ります。型は省略できず、本体は必ず `{ }` で囲んで `return` で値を返します。外側の変数を読み書きできます。

REWIND

```
fn main()->Int {
    let add = |a: Int, b: Int| -> Int { return a + b; };
    Out.println(add(1, 2));

    var count = 0;
    let bump = || -> Unit { count += 1; };
    bump();
    bump();
    Out.println(count);
    return 0;
}
```

出力

```
3
2
```

関数を引数に渡す

関数を受け取る引数には、関数型 `fn(引数の型) -> 戻り値の型 effects { ... }` を書きます。名前付きの関数も、クロージャも渡せます。

REWIND

```
fn apply(f: fn(Int) -> Int effects {}, x: Int) -> Int effects {} {
    return f(x);
}

fn inc(n: Int) -> Int effects {} { return n + 1; }

fn main()->Int {
    Out.println(apply(inc, 4));
    Out.println(apply(|n: Int| -> Int { return n * 2; }, 4));
    return 0;
}
```

出力

```
5
8
```

渡す関数の効果は、受け取る側の型で許された範囲に収まっている必要があります。この例では `effects {}`、つまり「何もしない関数」だけを受け取ります。

再帰の深さには上限がある

関数の呼び出しの深さには上限があり、約 1,000 段を超えると、`ExecutionBudgetExceeded` で止まります。深い再帰を書くときは、ループに書き換えられないか考えてください。この上限は、第23章で扱う「実行予算」の一部です。

第6章 List・Map・イテレータ

List

`List<T>` は、要素の型が `T` の、長さを変えられる列です。添字は 0 から始まります。`[ ]` で作れます。空の `List` は、型を明示して `List<Int>()` と書きます。

REWIND

```
fn main()->Int {
    let values = List<Int>();
    values.add(3);
    values.add(5);
    values.add(7);
    values.set(1, 9);
    Out.println(values.len());
    Out.println(values[1]);        // values.get(1) と同じ
    Out.println(values.pop());     // 末尾を取り出す。空なら None
    var total = 0;
    for v in values { total += v; }
    Out.println(total);
    return 0;
}
```

出力

```
3
9
Some(7)
12
```

添字が範囲の外を指すと、`IndexOutOfBounds` で止まります。`None` が返るわけではありません。入力から得た添字を使うときは、先に `len()` と比べてください。要素の型が合っていることと、添字が正しいことは別の問題です。

要素の型は最初の要素で決まり、型の違う要素を混ぜると拒否されます。入れ子にもできます（`[[1, 2], [3]]`）。

Map

`Map<K,V>` は、キーと値の対応です。値は `get` で取り出し、結果は `Option` です。キーが無ければ `None` になります。

REWIND

```
fn main()->Int {
    let ages = {"ann": 31, "bob": 28};
    Out.println(ages.get("ann"));
    Out.println(ages.get("zed"));
    ages.set("zed", 40);
    ages.remove("bob");
    Out.println(ages.len());
    for key in ages.keys() { Out.println(key); }   // キーは昇順
    return 0;
}
```

出力

```
Some(31)
None
2
ann
zed
```

`ages["ann"]` と書いても `ages.get("ann")` と同じです。`Map` のキーは、必ず並び順が決まります。そのため、どの環境で動かしても反復の順序が変わらず、結果が再現できます。

変更できる値には持ち主がいる

`List` や `Map` は、中身を後から変えられる値です。REWIND では、このような値をむやみにコピーせず、「いま誰が持っているか」を言語が追跡します。たとえば次のようには書けません。

エラーになる例

```
fn main()->Int {
    let a = [1];
    let b = a;      // 同じ List を二か所から持てない
    return 0;
}
```

これが、第9章で説明する **所有権** の考え方です。いまは「List はこう代入できない」とだけ覚えておけば、先へ進めます。

イテレータ

`xs.iter()` は、`List` の **その時点の内容** を順に取り出すイテレータを作ります。元の `List` を後から変更しても、作ったイテレータには影響しません。

メソッド	働き
`map(f)`	各要素を `f` で変換する
`filter(f)`	`f` が `true` を返す要素だけ残す

メソッド	働き
`fold(初期値, f)`	畳み込む
`take(n)`	先頭の `n` 個だけにする
`enumerate()`	`(添字, 要素)` のタプルにする
`zip(他)`	二つの列を組にする
`collect()`	`List` に集める
`next()`	次の要素を `Option` で返す

REWIND

```
fn main()->Int {
    let xs = [1, 2, 3, 4];
    let evens = xs.iter()
        .filter(|n: Int| -> Bool { return n % 2 == 0; })
        .map(|n: Int| -> Int { return n * 10; })
        .collect();
    Out.println(evens.len());
    Out.println(evens.get(1));
    Out.println(xs.iter().fold(0, |a: Int, b: Int| -> Int { return a + b; }));
    return 0;
}
```

出力

```
2
40
10
```

イテレータに渡す関数は、すぐに実行される同期的なものでなければなりません。タスクを作る関数や `publish` を伴う関数は渡せません。

タプル

`(a, b, c)` は、型の違う値をひとまとめにするタプルです。要素は `.0`、`.1` で読みます。`let` で分解して受け取ることもできます。複数の値を返す関数によく使います。

REWIND

```
fn minmax(a: Int, b: Int) -> (Int, Int) effects {} {
    if a < b { return (a, b); }
    return (b, a);
}

fn main()->Int {
    let (low, high) = minmax(9, 4);
    let pair = minmax(1, 2);
    Out.println(low);
    Out.println(high);
    Out.println(pair.1);
    return 0;
}
```

出力

```
4
9
2
```

第7章 自分の型を定義する

REWIND で自分の型を作る方法は、`struct`、`record`、`enum` の三つです。最初に、`struct` と `record` の違いを押さえましょう。

	`struct`	`record`
フィールドの書き換え	できる	できない
コピー	できない（持ち主が一人）	自由にできる
`==`	同じ一つの値か	**中身が等しいか**
向いているもの	状態を持つ、変化していくもの	設定、座標、メッセージなど、値そのもの

record: 変わらないデータ

`record` は、作った後に変えられないデータです。値は、フィールドの宣言順に引数を渡して作ります。

REWIND

```
record Point { x: Int, y: Int }

fn main()->Int {
    let p = Point(3, 4);
    let q = p;                    // コピーしてよい
    Out.println(p.x + q.y);
    Out.println(p == Point(3, 4));
    return 0;
}
```

出力

```
7
true
```

フィールドを書き換えようとすると、実行前にエラーになります。

エラーになる例

```
record Point { x: Int, y: Int }

fn main()->Int {
    let p = Point(3, 4);
    p.x = 1;
    return 0;
}
```

変更したいときは、新しい値を作ります。`record` は小さくて安いので、作り直しを恐れる必要はありません。

`record` のフィールドの型は、コピーして共有できる型（`Int`、`String`、他の `record` など）に限られます。`List` のような変更できる値を持たせたいときは、`freeze` した `Frozen<List<T>>` にします。

REWIND

```
record Config { names: Frozen<List<String>>, limit: Int }

fn main()->Int {
    let c = Config(freeze(["a", "b"]), 3);
    Out.println(c.names.len());
    return 0;
}
```

出力

```
2
```

`freeze` は第9章で説明します。

struct: 変えられるデータ

`struct` は、フィールドを後から書き換えられます。

REWIND

```
struct Counter { value: Int }

fn main()->Int {
    let c = Counter(1);
    c.value = 2;
    c.value += 5;
    Out.println(c.value);
    return 0;
}
```

出力

```
7
```

`let c` と書いていても、フィールドは更新できます。`let` が禁じるのは `c` に別の値を代入し直すことだけで、`c` の中身の更新とは別の話です。`struct` は `List` と同じく **持ち主が一人の値** で、コピーできません。第9章で、その扱い方を学びます。

> **メモ**: どちらを選ぶか迷ったときは、`record` から始めてください。変化していく状態が必要になったときに `struct` を選べば十分です。

enum: 選択肢を表す

`enum` は、「いくつかの形のうちのどれか」を表す型です。それぞれの形を **variant** と呼びます。variant は、値を持たないもの、位置で値を持つもの、名前付きのフィールドを持つものを、混ぜて宣言できます。値は `型名::variant` で作ります。

REWIND

```
enum Message {
    Quit,
    Write(String),
    Move{x: Int, y: Int}
}

fn describe(m: Message) -> String effects {} {
    match m {
        Message::Quit => { return "quit"; },
        Message::Write(text) => { return "write " + text; },
        Message::Move{x: a, y: b} => { return "move ${a},${b}"; }
    }
}

fn main()->Int {
    Out.println(describe(Message::Quit));
    Out.println(describe(Message::Write("hi")));
    Out.println(describe(Message::Move{x: 1, y: 2}));
    return 0;
}
```

出力

```
quit
write hi
move 1,2
```

match で網羅を検査する

`enum` に対する `match` は、**すべての variant を扱わなければなりません。**1 つでも漏れていると、実行前に拒否されます。あとから variant を足したとき、扱いを直し忘れた場所がすべてエラーになるので、見落としを防げます。

エラーになる例

```
enum Light { Red, Yellow, Green }

fn name(l: Light) -> String effects {} {
    match l {
        Light::Red => { return "赤"; },
        Light::Yellow => { return "黄"; }
    }
}

fn main()->Int { return 0; }
```

パターンには、次のものが使えます。

パターン	例	一致するもの
リテラル	`1`、`"a"`、`true`	等しい値
範囲	`0..5`	0 以上 5 未満の整数
名前	`v`	何にでも一致し、`v` に束縛する
`_`	`_`	何にでも一致し、捨てる
variant	`Some(x)`、`Shape::Rect(w, h)`	その variant。中身は入れ子のパターン
名前付きフィールド	`Message::Move{x: a, y: b}`	フィールドを持つ variant
`record`、`struct`	`Point{x: a, y: b}`	各フィールド
タプル	`(1, s)`	要素ごとのパターン

アームには `if 条件` の **ガード** を付けられます。ガード付きのアームは、網羅の根拠としては数えられません。

REWIND

```
enum Shape {
    Circle(Int),
    Rect(Int, Int),
    Empty
}

fn area(s: Shape) -> Int effects {} {
    match s {
        Shape::Circle(r) => { return 3 * r * r; },
        Shape::Rect(w, h) if w == h => { return w * w; },   // 正方形
        Shape::Rect(w, h) => { return w * h; },
        Shape::Empty => { return 0; }
    }
}

fn main()->Int {
    Out.println(area(Shape::Circle(2)));
    Out.println(area(Shape::Rect(3, 3)));
    Out.println(area(Shape::Rect(2, 5)));
    return 0;
}
```

出力

```
12
9
10
```

型にメソッドを足したいとき

`struct` や `record` に、直接メソッドを書く構文はありません（`impl Point { ... }` のような書き方は使えません）。メソッドは、次の章で学ぶ `trait` を通して定義します。値を作る関数や、値を操作する関数は、普通の関数として書けば十分です。

公開と非公開

型・関数・定数に `pub` を付けると、他の module から使えます。`struct` や `record` のフィールドに `private` を付けると、その module の外からは読み書き・構築・分解ができなくなります。値の中身を隠して、決まった関数を通してだけ扱わせたいときに使います。詳しくは第14章です。

第8章 trait とジェネリクス

trait: 型が持つ約束

`trait` は、「この型はこういう操作ができる」という約束です。メソッドの宣言を並べ、`impl トレイト for 型` で、それぞれの型について実装します。

メソッドの最初の引数は `self` で、その型は次のどれかです。

- `&Self`：読むだけで借りる
- `&mut Self`：変更するために借りる
- `Self`：所有権を受け取る

`trait` のメソッドには、**既定の実装** を付けられます。実装する側は、既定のあるメソッドを省略できます。

REWIND

```
trait Shape {
    fn area(self: &Self) -> Int effects {};
    fn describe(self: &Self) -> String effects {} {          // 既定の実装
        return "面積 " + self.area().format();
    }
}

record Square { side: Int }

impl Shape for Square {
    fn area(self: &Square) -> Int effects {} { return self.side * self.side; }
}

fn main()->Int {
    let sq = Square(3);
    Out.println(sq.area());
    Out.println(sq.describe());
    return 0;
}
```

出力

```
9
面積 9
```

メソッドは `値.メソッド()` の形で呼びます。`Int` のような組み込みの型にも実装できます。

ジェネリクス

型や関数は、**型引数** を取れます。`<T>` で宣言し、中身の型に関わらず使える部品を作れます。

REWIND

```
struct Stack<T> { items: List<T> }

fn main()->Int {
    let s = Stack<Int>(List<Int>());
    s.items.add(1);
    s.items.add(2);
    Out.println(s.items.len());
    return 0;
}
```

出力

```
2
```

型引数が引数などから決まるときは、省略できます。決まらないときは `List<Int>()` のように明示します。

型引数に制約を付ける

`T` が何でもよいとは限りません。たとえば二つの値のうち大きい方を返したいなら、`T` は大小を比べられる型でなければなりません。このような要求を、`T: 制約` で書きます。

制約	意味
`Share`	コピーして共有できる型
`Send`	タスクへ渡せる型
`Ord`	大小を比べられる型（`Map` のキーや並べ替えに使える）
`Eq`	等しいかどうか比べられる型
`Hash`	ハッシュ値を計算できる型
`Display`	文字列として表示できる型

制約は、`T: Ord + Share` のように `+` でつなぐか、`where` 節にまとめて書きます。

REWIND

```
fn largest<T>(a: T, b: T) -> T effects {} where T: Ord + Share {
    if a.cmp(b) >= 0 { return a; }
    return b;
}

fn main()->Int {
    Out.println(largest(3, 9));
    Out.println(largest("a", "b"));
    return 0;
}
```

出力

```
9
b
```

`Int` と `String` は、どちらも `Ord` を満たしているので、同じ関数に渡せます。

自作の型を比べる

独自の型を `Map` のキーにしたり、並べ替えたりするには、その型に `Ord` を実装します。`cmp` は、小さければ負、等しければ 0、大きければ正の整数を返します。

REWIND

```
record Version { major: Int }

impl Ord for Version {
    fn cmp(self: Self, other: Self) -> Int effects {} {
        if self.major < other.major { return -1; }
        if self.major > other.major { return 1; }
        return 0;
    }
}

fn main()->Int {
    let m = Map<Version, String>();
    m.set(Version(2), "two");
    m.set(Version(1), "one");
    Out.println(m.len());
    Out.println(Version(2).cmp(Version(1)));
    return 0;
}
```

出力

```
2
1
```

型引数付きの enum

`enum` も、型引数を取れます。標準の `Option<T>` や `Result<T,E>` は、このようにして作られている型です。自分で作ることもできます。

REWIND

```
enum Tree<T> { Leaf, Node(T) }

fn main()->Int {
    let t: Tree<Int> = Tree::Node(5);
    match t {
        Tree::Leaf => { Out.println("葉"); },
        Tree::Node(v) => { Out.println(v); }
    }
    return 0;
}
```

出力

```
5
```

第9章 所有権と借用

なぜ所有権があるのか

`List` や `struct` のように中身を変えられる値は、二か所から同時に持てると困ったことになります。片方で変更したつもりが、もう片方から見えてしまうからです。REWIND は、変更できる値を **暗黙にはコピーも共有もしません。**「誰がその値を持っているか」を型検査が追跡し、危ない書き方を実行前に拒否します。

巻き戻しとの関係もあります。保存した状態へ戻るときに、同じ値を指す名前が複数あると、「どの名前から見た状態が戻るのか」があいまいになります。持ち主を一人に決めておけば、戻る対象は常にはっきりします。

二種類の値

REWIND の値は、次の二種類に分かれます。

種類	例	扱い
共有値	`Int` 、`Float` 、`Bool` 、`String` 、`Bytes` 、`record` 、`Frozen<T>` 、タプルなど	自由にコピーして共有できる
変更できる値	`List` 、`Map` 、`struct` 、イテレータ、ファイルなどの資源	持ち主が一人。コピーできない

`let` か `var` かは、この区別とは関係ありません。`let xs = [1];` でも `xs.add(2)` は書けます。`let` が禁じるのは、`xs` に別の値を代入し直すことだけです。

共有値は、普通にコピーして使えます。難しいのは、変更できる値の扱いです。選べる方法は三つあります。

1. 所有権を渡す: `move`
2. 一時的に貸す: `&` （読むだけ）、`&mut` （変更する）
3. 変更できない複製を作って共有する: `freeze`

move: 持ち主を替える

`move 式` は、値の持ち主を新しい側に替えます。渡した後の元の名前を使うと、エラーです。

REWIND

```
fn count(xs: List<Int>) -> Int effects {} {
    return xs.len();
}

fn main()->Int {
    let xs = [1, 2, 3];
    Out.println(count(move xs));
    return 0;
}
```

出力

```
3
```

エラーになる例

```
fn count(xs: List<Int>) -> Int effects {} { return xs.len(); }

fn main()->Int {
    let xs = [1, 2, 3];
    let n = count(move xs);
    Out.println(xs.len());   // 渡した後は使えない
    return 0;
}
```

変更できる値を `move` なしで関数へ渡すと、`passing a mutable owner requires move or freeze` と拒否されます。「渡すと自分は使えなくなる」ことを、書き手に意識させるためです。

借用: 貸すだけで、持ち主は変わらない

関数に所有権を渡したくないときは、借用を使います。引数の型を `&T`（読むだけ）または `&mut T`（変更する）と宣言し、呼ぶ側も同じ記号を付けます。

REWIND

```
fn total(values: &List<Int>) -> Int effects {} {
    var sum = 0;
    for i in 0..values.len() { sum += values.get(i); }
    return sum;
}

fn append(values: &mut List<Int>, n: Int) -> Unit effects {} {
    values.add(n);
}

fn main()->Int {
    let xs = List<Int>();
    append(&mut xs, 7);
    append(&mut xs, 8);
    Out.println(total(&xs));
    return 0;
}
```

出力

```
15
```

借用の規則は、次の通りです。

- `&` の借用は、同時にいくつでも持てます。借用している間、持ち主は変更できません。
- `&mut` の借用は、同時に一つだけです。
- `&` の借用からは、値を変更できません。
- 借用を、関数の戻り値にしたり、タスクへ渡したりすることはできません。

エラーになる例

```
fn bad(xs: &List<Int>) -> Unit effects {} {
    xs.add(1);
}

fn main()->Int { return 0; }
```

エラーになる例

```
fn f(a: &mut List<Int>, b: &mut List<Int>) -> Unit effects {} {}

fn main()->Int {
    let xs = [1];
    f(&mut xs, &mut xs);
    return 0;
}
```

`xs.add(1)` や `xs.len()` のようなメソッド呼び出しでは、`&` や `&mut` は自動で付くので、書く必要はありません。

freeze と thaw: 変えない複製を共有する

`freeze(値)` は、その時点の内容を **変更できない共有用の複製** にします。型は `Frozen<T>` です。`Frozen` は共有値なので、コピーしても、`record` のフィールドに入れてもかまいません。読み取り（`len`、`get`、`for`）はできますが、変更するとエラーです。

`thaw(frozen)` は、元の `Frozen` と独立した、変更できる値を作ります。

REWIND

```
fn sum(xs: Frozen<List<Int>>) -> Int effects {} {
    var total = 0;
    for x in xs { total += x; }
    return total;
}

fn main()->Int {
    let snapshot = freeze([1, 2, 3]);
    Out.println(sum(snapshot));
    Out.println(sum(snapshot));      // 何度でも渡せる

    let copy = thaw(snapshot);
    copy.add(4);
    Out.println(copy.len());
    Out.println(snapshot.len());     // 元は変わらない
    return 0;
}
```

出力

```
6
6
4
3
```

エラーになる例

```
fn main()->Int {
    let s = freeze([1]);
    s.add(2);
    return 0;
}
```

`freeze` は、複製を作る分の費用がかかります。関数値や、ファイル・接続のような資源、循環する値は `freeze` できません。

使い分けの目安

- 関数が値を **見るだけ**: `&`
- 関数が値を **更新する**: `&mut`

- 関数が値を **受け取って持ち続ける**（保存、別のタスクへ渡すなど）: `move`
- 同じ値を **複数の場所で読みたい**: `freeze`

第10章 エラーを扱う

二種類の失敗

REWIND の失敗は、扱い方で二つに分かれます。

- **回復できる失敗**: 入力の誤りや、見つからない値など、呼び出し側が選んで処理すべきもの。`Option` や `Result` で返します。
- **致命的な失敗**: プログラムの誤りや、資源の限度超過など。`panic`、`assert` の失敗、リストの範囲外アクセス、ゼロ除算、整数のオーバーフロー、実行予算の超過がこれにあたります。実行が止まり、診断が表示されます。

「ユーザーが入力を間違えた」は前者、「プログラムのつじつまが合わなくなった」は後者です。前者を致命的な失敗にしてしまうと、呼び出し側は再入力を促すこともできません。

Option: あるか、ないか

`Option<T>` は、`Some(値)` か `None` のどちらかです。値が無いかもしれない場面で使います。`Map.get` や `List.pop` が返すのも `Option` です。

取り出すときは、`match`、`if let`、`let … else` を使います。失敗を無視して取り出す `unwrap` のようなメソッドはありません。「無いかもしれない」ことを、必ず書き手が処理することになります。

標準ライブラリの `std.option` には、無いときの既定値を指定して取り出す `valueOr` があります。

REWIND

```
import std.option as option;

fn main()->Int {
    let missing: Option<Int> = None;
    Out.println(option.valueOr(missing, 7));
    Out.println(option.valueOr(Some(3), 7));
    return 0;
}
```

出力

```
7
3
```

Result: 成功か、失敗か

`Result<T,E>` は、成功の `Ok(値)` か、失敗の `Err(エラー)` のどちらかです。文字列を整数にする `parseInt` のように、失敗しうる処理が返します。

REWIND

```
fn main()->Int {
    match "12".parseInt() {
        Ok(n) => { Out.println(n + 1); },
        Err(e) => { Out.println(e); }
    }
    match "x".parseInt() {
        Ok(n) => { Out.println(n + 1); },
        Err(e) => { Out.println(e); }
    }
    return 0;
}
```

出力

```
13
InvalidIntOrOverflow
```

? で失敗を呼び出し元へ返す

`Result` を返す関数の中で、`式?` と書くと、`Ok(v)` なら `v` を取り出し、`Err(e)` ならその場で関数から `Err(e)` を返します。失敗を伝えるだけの定型の処理を、一文字で書けます。

REWIND

```
fn parse(text: String) -> Result<Int, String> effects {} {
    return text.parseInt();
}

fn sum(a: String, b: String) -> Result<Int, String> effects {} {
    let x = parse(a)?;
    let y = parse(b)?;
    return Ok(x + y);
}

fn main()->Int {
    Out.println(sum("1", "2"));
    Out.println(sum("1", "x"));
    match sum("4", "5") {
        Ok(v) => { Out.println(v); },
        Err(e) => { Out.println(e); }
    }
    return 0;
}
```

出力

```
Ok(3)
Err(InvalidIntOrOverflow)
9
```

`?` は `Option` には使えません。`Option` は、`match` か `let … else` で扱います。

`?` を使う関数では、`Err` の型が、戻り値の `Err` の型と一致している必要があります。違うときは、`mapErr` で型を変えてから `?` を付けます。

REWIND

```
fn lookup(key: String) -> Result<Int, String> effects {} {
    if key == "a" { return Ok(1); }
    return Err("missing");
}

fn code(key: String) -> Result<Int, Int> effects {} {
    let v = lookup(key).mapErr(|e: String| -> Int { return e.byteLen(); })?;
    return Ok(v);
}

fn main()->Int {
    Out.println(code("a"));
    Out.println(code("b"));
    return 0;
}
```

出力

```
Ok(1)
Err(7)
```

`Result` には、他に `map`（`Ok` の中身を変換）、`andThen`（`Ok` の中身から次の `Result` を作る）、`flatten`（入れ子を一つにする）があります。

> **注意**: `?` のエラー型の不一致は、現在のところ、実際に `Err` の経路を通ったときの実行時エラーとして見つかります。実行前には拒否されません。`mapErr` で型をそろえる書き方を習慣にしておくと安全です。

panic と assert

`panic` と `assert` は、「ここに来たらプログラムの誤り」という前提を表すためのものです。

呼び出し	働き
`panic("メッセージ")`	実行を止める
`assert(条件)`	偽なら `AssertionFailed` で止まる

呼び出し	働き
`assert_eq(a, b)`	等しくなければ `AssertionFailed` で止まる（左右の値が表示される）

`assert_eq` の等しさは、`==` と同じ規則です。`List` どうしは、中身が同じでも別々の値なので、そのままでは失敗します。比べたいときは `freeze` します。

利用者の入力のように、起こりうる誤りには、`panic` や `assert` ではなく `Result` を使ってください。なお、`main` の途中で止まったときは、それまでに溜まっていた保留中の出力は外へ出ません。

テストを書く

`test fn` で書いた関数は、通常の実行では呼ばれず、`rewind test` で実行されます。第22章で詳しく扱いますが、書き方だけ見ておきましょう。

REWIND

```
fn double(x: Int) -> Int = x * 2;

test fn doubles() {
    assert_eq(double(4), 8);
}

fn main()->Int {
    Out.println(double(21));
    return 0;
}
```

出力

```
42
```

第11章 チェックポイント: commit と revert

ここからが、REWIND でいちばん特徴的な部分です。

保存して、戻る

`commit 名前;` は、いまの計算状態に名前を付けて保存します。この保存点を **チェックポイント** と呼びます。`revert 名前;` は、その保存点の状態に戻します。

次の例は、送金の途中で残高不足に気づいたとき、書き換えを取り消します。

REWIND

```
fn main()->Int {
    let balances = [100, 50];          // 口座 0 と口座 1
    commit start;                      // 送金前の状態を保存する

    balances.set(0, balances.get(0) - 130);
    balances.set(1, balances.get(1) + 130);

    if balances.get(0) < 0 {
        revert start;                  // 残高不足。書き換えをなかったことにする
        Out.println("残高不足のため取り消しました");
    }
    Out.println(balances.get(0));
    Out.println(balances.get(1));
    return 0;
}
```

出力

```
残高不足のため取り消しました
100
50
```

自分で「元の値」を取っておいて書き戻す必要はありません。何をどれだけ書き換えていても、`revert` が保存時の状態にまとめて戻します。

何が戻るのか

`revert` で戻るのは、プログラムの **計算状態** と、まだ外に出ていない **保留中の出力** です。

戻る	戻らない
変数の値、`List` や `Map` の中身	`publish` で外に出した出力

戻る	戻らない
実行中のタスクの状態	すでに送信した通信、確定したデータベースの書き込み
保留中の出力（`Out.println` など）	

次の例で、保留中の出力が取り消される様子を確かめましょう。

REWIND

```
fn main()->Int {
    let items = [1, 2];
    var n = 10;
    commit saved;

    items.add(3);
    n = 99;
    Out.println(n);          // 保留中のまま、revert で消える

    revert saved;
    Out.println(items.len());
    Out.println(n);
    drop saved;
    return 0;
}
```

出力

```
2
10
```

`99` は、出力されません。「何を戻せて何を戻せないか」の境目は、次の第12章でじっくり見ます。

revert は、実行位置を戻さない

見落としやすい点です。`revert` が戻すのは **状態だけ** で、プログラムの実行位置は動きません。`revert` の次の文から、そのまま実行が続きます。上の例でも、`revert saved;` の次の `Out.println(items.len());` が実行されています。

実行位置まで戻したいときは、`resume 名前;` を使います。`resume` は、状態と実行位置を、`commit 名前;` の直後へ戻します。

resume の注意: 終わらないループ

`resume` は状態も戻すので、「状態だけを脱出条件にしたループ」は決して終わりません。

エラーになる例

```
fn main()->Int {
    var n = 0;
    commit start;
    n += 1;                  // resume で n は 0 に戻る
    if n < 3 { resume start; }
    return 0;
}
```

この例では、`n` が `resume` のたびに 0 へ戻るため、`n < 3` はいつまでも真です。実行は、第23章で説明する「実行予算」を使い切って止まります。

脱出条件には、**巻き戻らないもの** を使います。公開済みの結果、消費した実行予算、外部領域で記録された観測（第12章）などです。そのため、日常的なコードでは `resume` より `revert` を使う場面の方がずっと多くなります。

保存点の名前と寿命

チェックポイントの名前は、変数や関数の名前とは別の名前空間にあります。次の規則があります。

- 同じ名前で二度 `commit` することはできません。
- 存在しない名前に `revert` や `drop` するとエラーです。
- `begin` は予約名です（第12章）。
- 不要になった保存点は `drop 名前;` で捨てられます。保存点は、保存した状態の分だけメモリを使います。使い終わったら捨てると、メモリを解放できます。

`revert` で戻れるのは、**同じ関数の中の、まだ有効なスコープにある保存点** です。呼び出し元の関数が作った保存点や、すでに終わった内側のスコープの保存点へ戻ろうとすると、`InvalidContinuation` になります。

エラーになる例

```
fn undo() -> Unit effects {} {
    revert outer;            // 呼び出し元で作った保存点には戻れない
}

fn main()->Int {
    commit outer;
    undo();
    return 0;
}
```

保存点より後に作った変数は、`revert` で消えます。

考え方のまとめ

- `commit` は、**外へ確定する操作ではありません。** あとで戻るための保存点を作るだけです。

- `revert` は、状態と保留中の出力を戻します。実行位置は動きません。
- 外へ出す（確定する）ためには、別の操作 `publish` を使います。次の章で学びます。

第12章 公開と外部領域

三つの層

実行中のものは、次の三つの層に分けて考えると整理できます。

層	内容	`revert` で
計算状態	変数、`List` や `Map` の中身、タスク、入力の読み取り位置、乱数の状態	戻る
保留中の入出力	`Out.println` や `File.writeText` の結果で、まだ外に出ていないもの	取り消される
確定済みの結果	`publish` で外へ出たもの	戻らない

`Out.println("hi")` を実行しても、すぐ画面に出るわけではありません。いったん保留中になり、「確定してよい」と宣言されたときに初めて外へ出ます。この宣言が `publish;` です。

publish: 外へ確定する

`publish;` は、保留中の入出力を外へ出します。公開した後の `revert` が戻せるのは、公開より **後の** 変化だけです。公開済みの結果は戻らず、再び送られることもありません。

REWIND

```
fn main()->Int effects {output} {
    var score = 10;
    commit first;
    score = 99;
    Out.println(score);
    publish;                // 99 が外へ出る
    revert first;
    Out.println(score);     // score は 10 に戻った。この出力は保留中
    return 0;               // main が戻るときに公開される
}
```

出力

```
99
10
```

`99` は取り消せず、表示されたままです。

main が戻ったときの自動公開

`main` が戻ると、保留中の出力は **自動で公開** されます。普通のプログラムでは、`publish;` を一度も書く必要はありません。`panic` や失敗で止まったときは公開されません。途中までの出力が外へ漏れることなく、「成功したときだけ出力が出る」という性質が得られます。

`publish;` を明示するのは、次のようなときです。

- 入力を待つ前に、プロンプトを見せたい（`In.readLine()` の前）。
- 長い処理の途中で、結果を外へ確定しておきたい。
- 取り消したくない結果を確定してから、先へ進みたい。

公開は、複数のファイルやストリームを **一度にまとめて** 適用する保証がありません。適用の途中で失敗すると `PublishPartiallyApplied` になり、どこまで適用されたかが診断に出ます。

begin: 最初の状態へ戻る

プログラムの実行前に、`begin` という名前のチェックポイントが自動で作られます。`revert begin;` は、ここまでに作った変数やチェックポイントも含めて、すべてを開始時の状態に戻します。

REWIND

```
fn main()->Int {
    var x = 1;
    commit a;
    x = 2;
    Out.println("one");
    publish;                // one は確定済み
    x = 3;
    Out.println("two");     // 保留中
    revert begin;           // x も a も消え、two も消える
    Out.println("after");
    return 0;
}
```

出力

```
one
after
```

`begin` は予約名で、自分で `commit` や `drop` はできません。変数や関数の名前にも使えません。

branch: 試すだけ試す

`branch 名前 { ... }` は、ブロックの中を試しに実行し、**終わると状態を実行前に戻します**。「もしこうしたら、どうなるか」を、状態を汚さずに確かめたいときに使います。ブロックの中の保留中の出力も捨てられますが、`publish` した結果は残ります。

REWIND

```
fn total(cart: &List<Int>) -> Int effects {} {
    var sum = 0;
    for i in 0..cart.len() { sum += cart.get(i); }
    return sum;
}

fn main()->Int {
    let cart = [300, 500];
    branch preview {
        cart.add(1200);                                  // もし高い品を足したら?
        Out.println("試算: ${total(&cart)} 円");
        publish;                                         // 試算の結果だけ外へ出す
    }
    Out.println("実際: ${total(&cart)} 円");
    return 0;
}
```

出力

```
試算: 2000 円
実際: 800 円
```

ブロックの終わりで `cart` は元に戻っています。

外部領域: 取り直せない値を扱う

時刻やネットワークの応答のように、外の世界から得た値は、巻き戻した後に取り直すと別の値になりえます。そうなると、「同じ状態へ戻って、もう一度実行する」という前提が崩れてしまいます。

REWIND は、そのような操作を `external { ... }` という **外部領域** に閉じ込めます。領域の中の操作の結果は **記録** され、巻き戻して実行し直したときは、時計やネットワークを再び見に行かず、記録された結果が使われます。

REWIND

```
import std.external as host;

fn main()->Int effects {external, clock, fileRead, fileWrite, output} {
    var stamp = 0;
    commit before;
    external {
        match host.millis() {
            Ok(n) => { stamp = n; },
            Err(_) => {}
        }
    }
    File.writeText("first.txt", stamp.format());
    publish;                        // 一度目の値をファイルに確定する
    revert before;                  // stamp は 0 に戻る
    external {
        match host.millis() {
            Ok(n) => { stamp = n; },
            Err(_) => {}
        }
    }
    match File.readText("first.txt") {
        Ok(text) => { Out.println(text == stamp.format()); },
        Err(_) => { Out.println("読めない"); }
    }
    return 0;
}
```

出力

```
true
```

二度目の `host.millis()` は、時計を読み直さず、一度目に記録した値を返します。そのため、巻き戻す前後で値が一致します。

領域には三つの形があります。

形	意味
`external { ... }`	結果を記録し、巻き戻した後は記録を再利用する
`external fresh { ... }`	いつも新しく実行し、その結果を記録する
`external live { ... }`	物理的な操作を実行する。巻き戻しや再生では再利用しない。`live` 効果の許可が必要

領域の中では、外部操作だけを行います。`await`、`publish`、`commit`、`revert`、`return`、`break`、`continue` は使えません（`ExternalBoundary`）。通信を領域の中で始め、領域を出てから `await` で結果を受け取る、という使い方になります。

データベースへの書き込みのように、すでに外の世界へ及んだ作用は、`revert` で取り消せません。外へ出る操作は「戻せない」ことを前提に設計してください。

第13章 効果と許可

効果とは何か

効果（effect）は、関数が外の世界に対して行う操作の種類です。画面への出力、ファイルの読み書き、現在時刻の取得、通信などがあたります。REWIND では、関数が行う効果を `effects { ... }` で宣言します。宣言に無い効果を使うプログラムは、実行する前に拒否されます。

エラーになる例

```
fn greet() -> Unit effects {} {
    Out.println("hi");        // output を宣言していない
}

fn main()->Int {
    greet();
    return 0;
}
```

何もしない関数は `effects {}` と書きます。これは、「この関数は外部とやりとりしない。同じ引数なら同じ結果になる」という約束です。この約束があるおかげで、巻き戻しや再生をしても安全だと言えます。

効果の一覧

効果	できること
`output`	`Out` 、`Err` への出力
`input`	`In` からの入力
`args`	`Args` による起動引数の読み取り
`env`	`Env` による環境変数の読み取り
`locale`	`Locale` による地域設定の読み取り
`clock`	`Time` による時刻の観測
`random`	`Random` による乱数
`tasks`	タスクとチャネルの作成・待機
`fileRead`	`File` の読み取り、`Directory` の一覧
`fileWrite`	`File` 、`Directory` の書き込み・作成・削除
`external`	`external { }` での外部操作
`live`	`external live { }` での、再利用しない物理操作

効果	できること
`network`	HTTP、TCP などの通信
`db`	SQLite、PostgreSQL
`gui`	ネイティブ画面

この表にない名前を宣言すると `unknown effect` になります。

効果は呼び出し元へ伝わる

関数が呼ぶ関数の効果は、呼ぶ側も宣言しなければなりません。

REWIND

```
fn greet() -> Unit effects {output} {
    Out.println("hi");
}

fn twice() -> Unit effects {output} {
    greet();
    greet();
}

fn main()->Int effects {output} {
    twice();
    return 0;
}
```

出力

```
hi
hi
```

どこかで宣言が足りないと、不足している箇所と、その原因となった呼び出しの経路が、`caused by` として表示されます。どこで宣言を足せばよいかが一目で分かります。

`main` だけは `effects` を省略でき、そのときは本体が必要とする効果が自動で求められます。ここまでの例で `main` に `effects` を書かなかったのは、そのためです。

宣言と許可は別

効果の宣言は、「この関数は何をするか」の表明です。それを実際に **許す** かどうかは、プログラムを **実行する人** が決めます。宣言だけでは何も許可されません。

実行のしかた	許可される効果
単体のファイルの `rewind run`	既定で `input`、`output`、`args`、`locale`、`random`、`tasks`

実行のしかた	許可される効果
さらに許可する	`--allow-effects fileRead,fileWrite` のように、カンマ区切りで指定する
プロジェクト	`rewind.toml` の `effects`

たとえばファイルを書くプログラムは、`fileWrite` を宣言したうえで、次のように実行します。

コマンド

```
rewind run main.rw --allow-effects fileRead,fileWrite
```

許可が無いと、`effect gui is not allowed` のように拒否されます。他人が書いたプログラムを実行するときも、何を許すかを自分で決められるので、安心して試せます。

環境変数は、効果 `env` に加えて **変数名ごとの許可** が必要です。`--allow-env 名前` で許可します。値を診断や記録に残したくないときは `--secret-env 名前` を使います。

ファイルのパス

`File` と `Directory` のパスは、実行の **root** からの相対パスです。root は、単体のファイルではそのファイルのあるディレクトリで、プロジェクトでは `--root` で指定した場所です。区切りには、OS に関係なく `/` を使います。絶対パス、`..`、root の外を指すシンボリックリンクは、`InvalidPath` で拒否されます。

REWIND

```
fn main()->Int effects {fileRead, fileWrite, output} {
    File.writeText("state.txt", "saved");
    Out.println(File.readText("state.txt"));    // 保留中の書き込みも読める
    Out.println(File.readText("missing.txt"));
    Out.println(File.readText("../escape.txt"));
    return 0;
}
```

出力

```
Ok(saved)
Err(NotFound: missing.txt (file not found: missing.txt))
Err(InvalidPath: ../escape.txt (invalid path: ../escape.txt))
```

読み取りの失敗は `Result` で返ります。書き込みは保留中になり、`publish` か `main` の終了で、実際のファイルに反映されます。

Secret: 値を伏せる

`secret(値)` は、値を `Secret<T>` で包みます。`Secret` を出力すると `<redacted: Secret>` と表示され、診断や記録にも値が残りません。取り出すときは `reveal` を使います。パスワードやトークンを扱うときの、うっかり漏らさないための仕組みです。暗号化や、外部への送信の防止までは保証しません。

第14章 module とプロジェクト

import

標準ライブラリや、別のファイルの宣言を使うには `import` します。ファイルの先頭に書き、基本的に別名（`as`）を付けて使います。

REWIND

```
import std.number as number;
import std.text as text;

fn main()->Int {
    Out.println(number.integer("ff", 16));
    Out.println(text.length("日本語"));
    return 0;
}
```

出力

```
Ok(255)
3
```

書き方	意味
`import std.number as number;`	標準ライブラリの module を別名で使う
`import util as u;`	同じプロジェクトの `util.rw` を別名で使う
`import util.{double, total};`	名前を選んで直接取り込む
`import util.{double as dbl};`	取り込む名前に別名を付ける

別名を付けない `import util;` では、`util` という名前は使えるようになりません。必ず `as` か `{ }` を使います。

公開する名前を決める

ファイルの中の宣言は、既定では他のファイルから見えません。`pub` を付けた宣言だけが公開されます。公開できるのは、`fn`、`struct`、`record`、`enum`、`type`、`trait`、`const` です。公開する関数には、戻り値の型が必要です。

公開した `struct` のフィールドに `private` を付けると、内部の表現を隠したまま、公開した関数を通して扱わせられます。次は、二つのファイルから成る例です。

```
// util.rw
pub struct Account { private total: Int }
pub fn open() -> Account effects {} { return Account(0); }
pub fn deposit(a: &mut Account, n: Int) -> Unit effects {} { a.total += n; }
pub fn balance(a: &Account) -> Int effects {} { return a.total; }
pub const LIMIT: Int = 99;

// main.rw
import util as u;
fn main()->Int {
    let a = u.open();
    u.deposit(&mut a, 5);
    Out.println(u.balance(&a));   // 5
    Out.println(u.LIMIT);         // 99
    // a.total は module の外から読めない
    return 0;
}
```

公開されていない関数を呼ぶと、`unknown name` と報告されます。非公開のフィールドを読むと `field total is private to its module` です。

こうして、「内部の都合を隠して、外には決まった窓口だけを見せる」設計ができます。`Account` の内部表現を変えても、公開した関数が同じ働きをする限り、使う側のコードは変わりません。

module の中の状態

module の最上位に書けるのは宣言だけです。`const` を除いて、module が **状態を持つことはできません**。状態は、値として作って、引数や戻り値で受け渡します。巻き戻しの対象が、いつも「いま動いている関数の中の値」に限られるのは、このためです。

プロジェクトにする

ファイルが複数あるプログラムや、権限・依存関係を固定したいプログラムは、ディレクトリに `rewind.toml` を置いてプロジェクトにします。

```
language = "3.6.0"
source_root = "src"
entry = "main.rw"
effects = "output,fileRead"
```

キー	意味
`language`	言語バージョン
`source_root`	ソースのあるディレクトリ
`entry`	実行を始めるファイル

キー	意味
`effects`	プログラムに許可する効果
`[assets]`	実行に同梱するファイル
`[dependencies]`	依存パッケージ。`rewind update` が `rewind.lock` を書く

マニフェストが近くにあると、その `language` と `effects` が使われます。単体のファイルを実行するときは、マニフェストも `rewind compile` も要りません。

第15章 並行処理

async fn と Task

`async fn` は、呼んでもすぐには実行されません。代わりに、実行の予約である `Task<T>` を返します。`await` で結果を待つと実行され、`spawn` で先に開始させることもできます。`await` は `Result<T, TaskError>` を返すので、タスクの失敗を呼び出し側で受け止められます。

REWIND

```
async fn compute(n: Int) -> Int effects {} {
    return n * 2;
}

fn main()->Int {
    let direct = compute(4);
    match await direct {
        Ok(v) => { Out.println(v); },
        Err(_) => { Out.println("失敗"); }
    }

    let started = spawn compute(21);
    match await started {
        Ok(v) => { Out.println(v); },
        Err(_) => { Out.println("失敗"); }
    }
    return 0;
}
```

出力

```
8
42
```

決定的な協調スケジューラ

REWIND のタスクは、OS のスレッドで並列に走るのではなく、**協調的なスケジューラ** の上で動きます。`await` などの切り替え点で、順番に実行が移ります。切り替えの順序は決まっているので、**同じ入力なら、実行するたびに同じ結果** になります。タイミングによって結果が変わる、というバグが生まれません。

これは巻き戻しや再生とも相性がよい設計です。タスクの状態も計算状態の一部なので、`revert` で戻せます。

タスクへ渡せる値

タスクに渡す引数は、`Send` でなければなりません。変更できる値は `move` で渡すか、`freeze` した値を渡します。借用はタスクへ渡せません。

REWIND

```
async fn total(xs: List<Int>) -> Int effects {} {
    return xs.len();
}

fn main()->Int {
    let xs = [1, 2, 3];
    match await spawn total(move xs) {
        Ok(n) => { Out.println(n); },
        Err(_) => {}
    }
    return 0;
}
```

出力

```
3
```

失敗・取り消し・時間切れ

`TaskError` には三つの場合があります。

- `Failed(診断)` : タスクの中で致命的な失敗が起きた
- `Cancelled` : `cancel()` で取り消された
- `TimedOut` : `timeout(n)` で指定したステップ数を超えた

時間切れは、実時間ではなく、タスクが実行した **論理的なステップ数** で数えます。どんなに遅い環境でも、同じ結果になります。

REWIND

```
async fn spin() -> Int effects {} {
    while true {}
    return 0;
}

async fn divide(a: Int, b: Int) -> Int effects {} {
    return a / b;
}

fn main()->Int {
    let t = spawn spin();
    t.cancel();
    match await t {
        Err(TaskError::Cancelled) => { Out.println("取り消し"); },
        _ => {}
    }

    match await spin().timeout(20) {
        Err(TaskError::TimedOut) => { Out.println("時間切れ"); },
        _ => {}
    }

    match await spawn divide(1, 0) {
        Err(TaskError::Failed(d)) => { Out.println(d.code); },
        _ => {}
    }
    return 0;
}
```

出力

```
取り消し
時間切れ
DivisionByZero
```

`t.isDone()` は、待たずに完了しているかを調べます。

チャネル

`Channel<T>(容量)` は、タスク間で値を受け渡す通路です。容量が 0 なら、送り手と受け手が出会うまで待ち合わせます。`send` と `receive` は `await` で待ち、`close()` で閉じます。閉じて空になると、`receive` は `Err` を返します。

REWIND

```
async fn producer(c: Channel<Int>) -> Unit effects {tasks} {
    for i in 0..3 {
        match await c.send(i) { Ok(_) => {}, Err(_) => {} }
    }
    c.close();
}

async fn consumer(c: Channel<Int>) -> Int effects {tasks} {
    var total = 0;
    while true {
        match await c.receive() {
            Ok(v) => { total += v; },
            Err(_) => { break; }
        }
    }
    return total;
}

fn main()->Int effects {tasks, output} {
    let c = Channel<Int>(1);
    let p = spawn producer(c);
    let r = spawn consumer(c);
    match await r {
        Ok(v) => { Out.println(v); },
        Err(_) => {}
    }
    return 0;
}
```

出力

```
3
```

TaskGroup: 片付け忘れを防ぐ

`using group = TaskGroup();` で作ったグループに、`group.add(タスク)` で加えたタスクは、グループのスコープを出るときに必ず片付けられます。

- 正常にスコープを出るときは、加えたタスクの終了を **待ちます**。
- `Result` を返す関数が `Err` を返して抜けるときは、終わっていないタスクを **取り消します**。

REWIND

```
async fn work(n: Int) -> Unit effects {output} {
    Out.println("work ${n}");
}

fn run() -> Unit effects {tasks, output} {
    using group = TaskGroup();
    group.add(work(1));
    group.add(work(2));
    Out.println("スコープの終わり");
}

fn main()->Int effects {tasks, output} {
    run();
    Out.println("戻った");
    return 0;
}
```

出力

```
スコープの終わり
work 1
work 2
戻った
```

`spawn` したタスクを一度も `await` せずに `main` が戻ると、そのタスクは実行されないことがあります。結果を使うタスクは、必ず `await` するか、`TaskGroup` に入れてください。

第16章 後始末: defer と using

defer

`defer` は、「現在のスコープを出るときにやること」を予約します。`return` で出るときも、失敗で止まるときも実行されます。複数予約したときは、**予約したのと逆の順** に実行されます。

REWIND

```
fn work() -> Unit effects {output} {
    defer || -> Unit { Out.println("後始末 1"); };
    defer || -> Unit { Out.println("後始末 2"); };
    Out.println("作業");
}

fn main()->Int {
    work();
    return 0;
}
```

出力

```
作業
後始末 2
後始末 1
```

逆順になるのは、資源を確保した順の逆に解放するのが安全だからです。後から確保したものが、先に確保したものに依存していることがよくあります。

ループの本体で `defer` を使うと、反復のたびに実行されます。`defer 式;` の形で、式をそのまま予約することもできます。後始末が失敗したときは、元の失敗を主な診断として残し、後始末の失敗は原因（`causes`）に追加されます。元の失敗が隠されることはありません。

using

`using 名前 = 式;` は、資源（ファイルハンドル、データベース接続、`TaskGroup` など）を、スコープを出るときに確実に閉じる宣言です。`defer` で閉じる処理を書くより、資源を作る行と閉じる動作が一か所にまとまります。

次は、ファイルを開いて途中から読む例です。関数を出るとき、`f` は自動で閉じられます。

REWIND

```
fn readTail() -> String effects {fileRead} {
    using f = File.openSnapshot("a.txt");
    f.seek(6);
    return f.read(5);
}

fn main()->Int effects {fileRead, fileWrite, output} {
    File.writeText("a.txt", "hello world");
    publish;
    Out.println(readTail());
    return 0;
}
```

出力

```
world
```

`File.openSnapshot` は、開いた時点のファイルの内容を読み出すハンドルを作ります。`seek` で位置を決め、`read(n)` で `n` 文字分を読みます。

巻き戻しと後始末

`using` や `defer` による後始末は、「スコープを抜けた」ことを意味するだけです。外部へ書いた内容が確定したことまでは意味しません。ファイルへ書き込んだ内容が外に出るのは、`publish` のときです（第12章）。

第17章 入力・引数・ファイル

この章では、プログラムが外の世界から値を受け取り、結果を外へ渡す方法を整理します。どの操作にも、第13章で学んだ効果の宣言が必要です。

標準入力から読む

`In.readLine()` は、標準入力から 1 行を読み、`Option<String>` を返します。入力の終わりに達すると `None` になります。したがって、終わりまで読む処理は、`None` を受け取るまで繰り返すループで書けます。

次のプログラムは、1 行ごとに整数を読んで合計します。数でない行は、標準エラーへ知らせて読み飛ばします。

REWIND

```
fn main()->Int effects {input, output} {
    var total = 0;
    var lines = 0;
    while true {
        match In.readLine() {
            None => { break; },
            Some(line) => {
                match line.parseInt() {
                    Ok(n) => { total += n; lines += 1; },
                    Err(_) => { Err.println("数ではない行: " + line); }
                }
            }
        }
    }
    Out.println("${lines} 行、合計 ${total}");
    return 0;
}
```

出力

```
3 行、合計 35
```

標準入力に `10`、`20`、`abc`、`5` の 4 行を渡して実行した結果です。端末では次のように試せます。

コマンド

```
printf '10\n20\nabc\n5\n' | rewind run main.rw
```

「入力が途中で終わった」ことを `None` として受け取り、足りないという扱いにするか、空の入力を許すかは、プログラムの仕様として決めてください。`None` を `0` とみなして加算してしまうと、入力の欠落に気づけません。

人に入力を促す対話型のプログラムでは、`In.readLine()` の前に `publish;` を書いて、プロンプトを先に表示します。出力は `main` が戻るまで保留中のままだからです。

大量の数値を読む

数百万個の数を読むとき、1 行ごとに `String` を作って分割すると、途中の値が大量にできます。`std.scanner` は、`Bytes` を保持したまま位置を進めながら数を読むので、中間の文字列を作りません。

REWIND

```
import std.scanner as scanner;

fn main()->Int {
    match scanner.fromBytes(Bytes("10 -3 7\n")) {
        Err(_) => { Out.println("作れません"); },
        Ok(scan) => {
            var total = 0;
            var going = true;
            while going {
                match scanner.nextInt(&mut scan) {
                    Ok(Some(n)) => { total += n; },
                    Ok(None) => { going = false; },     // 入力の終わり
                    Err(_) => { going = false; }        // 数として不正、または範囲外
                }
            }
            Out.println(total);
        }
    }
    return 0;
}
```

出力

```
14
```

`nextInt` は、成功したときは `Ok(Some(数))`、入力の終わりでは `Ok(None)` を返し、不正な数や範囲外のときは `Err` になります。空白は ASCII の空白文字として扱われ、Unicode の空白は区切りになりません。

起動引数

プログラムに渡す引数は、`Args.all()` で読みます。引数は `rewind` コマンドの `--` の後ろに書きます。手前に書いたものは `rewind` 自身のオプションです。`--` の後ろに書いた `--help` は、プログラムの引数として渡されます。

REWIND

```
fn main()->Int effects {args, output} {
    let args = Args.all();
    Out.println(args.len());
    if args.len() > 0 { Out.println(args.get(0)); }
    return 0;
}
```

出力

```
2
hello
```

コマンド

```
rewind run main.rw -- hello world
```

`get` は範囲外のとき止まるので、先に `len()` で数を確認します。使い方の説明を、範囲外のエラーに任せてはいけません。

ファイルの読み書き

ファイルの操作は、`File` と `Directory` を使います。

操作	働き
`File.readText(path)`	文字列で読む。`Result<String,FileError>`
`File.readBytes(path)`	バイト列で読む
`File.writeText(path, 文字列)`	書き込み（保留中）。無ければ作る
`File.append(path, 文字列)`	末尾へ追記（保留中）
`File.openSnapshot(path)`	読み出し用のハンドルを開く
`Directory.entries(path)`	名前の一覧

書き込みは、第12章で見たとおり **保留中** です。`publish` か `main` の終了で、実際のファイルに反映されます。保留中の書き込みは、同じプログラムの中から読み戻せます。

REWIND

```
fn main()->Int effects {fileRead, fileWrite, output} {
    File.writeText("log.txt", "first\n");
    File.append("log.txt", "second\n");
    publish;
    match File.readText("log.txt") {
        Ok(text) => { Out.println(text.split("\n").len()); },
        Err(e) => { Out.println(e); }
    }
    return 0;
}
```

出力

```
3
```

`"first\nsecond\n"` を `"\n"` で分けると、末尾の空文字列を含めて 3 つになるため、`3` と表示されます。

ファイルを読んで検査し、新しい内容を用意してから、最後に `publish` で書き込む、という流れは扱いやすい設計です。検査に失敗した場合は、`revert` や `main` の中断で、ファイルにはまったく触れずに済みます。

保存の直前に、別のプログラムがファイルを変更していたら、`ExternalStateConflict` になることがあります。黙って上書きするのではなく、利用者に競合を伝える設計を考えてください。

複数の書き込みは一括では確定しない

複数のファイルやディレクトリ、標準出力を、一つの `publish` にまとめても、OS 全体のトランザクションにはなりません。途中で失敗すると `PublishPartiallyApplied` になり、どこまで適用されたかが診断に出ます。「エラーなら何も起きていない」と考えて再実行すると、すでに保存したファイルを二重に適用するかもしれません。実際の外部の状態と診断を確かめてから、再開の方法を決めてください。

データベースのトランザクションとも別物です。ファイルの `publish` はデータベースの確定ではなく、データベースの確定は REWIND のチェックポイントの保存ではありません。この区別は、第24章でもう一度扱います。

時刻と乱数

時刻と乱数は、どちらも **外の世界から得る値** です。`Time.now()` は `clock` 効果、`Random.next()` は `random` 効果を必要とします。

実行を記録しておくと（第22章）、これらの値も一緒に記録され、再生するときは同じ値が使われます。乱数や時刻を使うプログラムでも、失敗した実行を同じ値で再現できます。

環境変数

環境変数は `Env.get(名前)` で読みます。効果 `env` に加えて、変数名ごとの許可（ `--allow-env 名前` ）が必要です。パスワードのような値は `Env.getSecret(名前)` で読むと、`Secret` として扱われ、診断や記録に値が残りません。

第18章 文字列・Unicode・正規表現

「一文字」の数え方は一つではない

文字列の長さや位置を扱うとき、REWIND は「何を単位に数えるか」を、メソッドごとにはっきり分けています。日本語や絵文字を含む文字列では、この違いが結果に表れます。

操作	位置・長さの単位
`s.byteLen()` 、`s.find(t)` 、`s.slice(a, b)`	UTF-8 のバイト
`s.charLen()` 、`std.text` の `length` 、`slice`	Unicode のスカラー値
`s.utf16Len()`	UTF-16 のコード単位
`std.unicode` の `grapheme…`	書記素クラスタ（人が見る一文字）
`std.regex` の範囲	UTF-8 のバイト位置

REWIND

```
import std.text as text;

fn main()->Int {
    let s = "a😀界";
    Out.println(s.byteLen());
    Out.println(s.charLen());
    Out.println(s.utf16Len());
    Out.println(text.slice(s, 1, 3));    // スカラー値の 1 以上 3 未満
    Out.println(s.slice(1, 5));          // バイト位置の 1 以上 5 未満
    return 0;
}
```

出力

```
8
3
4
Ok(😀界)
Ok(😀)
```

`"a😀界"` は、バイトで数えると 8（`a` が 1、`😀` が 4、`界` が 3）、スカラー値では 3、UTF-16 では 4 です。バイトの位置で文字の途中を指すと、`slice` は `Err` を返します。

単位を混ぜると、日本語や絵文字で不具合が出ます。たとえば、正規表現が返したバイト位置を、スカラー値の位置を受け取る `std.text.slice` へそのまま渡すと、別の場所を切り出してしまいます。単位を、関数名や変数名、ドキュメントに書き残す習慣をつけましょう。

書記素と正規化

目に見える一文字が、複数のスカラー値でできていることがあります。「e」に結合アクセントを重ねた文字や、国旗、家族の絵文字がそうです。人が見る一文字の単位で扱いたいとき（たとえば入力欄の Backspace）には、`std.unicode` の書記素クラスタを使います。

REWIND

```
import std.unicode as unicode;

fn main()->Int {
    let source = "éJP👪";
    Out.println(unicode.graphemeCount(&source));
    Out.println(unicode.graphemeSlice(&source, 1, 2));    // 2 つ目の一文字
    let word = "straße";
    Out.println(unicode.upper(&word));
    return 0;
}
```

出力

```
Ok(3)
Ok(JP)
Ok(STRASSE)
```

見た目が同じ文字列でも、内部のスカラー値の並びが違うことがあります（「é」は 1 つの文字としても、「e」と結合記号の 2 つとしても表せます）。これを同じとみなして比べたいときは、正規化形（ `NFC` 、 `NFD` 、 `NFKC` 、 `NFKD` ）を指定して正規化します。 `String` や `Map` のキーが、自動で正規化されることはありません。

互換正規化（ `NFKC` 、 `NFKD` ）は、見た目や意味の違う文字も同じにしてしまうことがあります。利用者が入力した原文を、そのまま書き換えてよいかどうかは、用途で判断してください。

正規表現

`std.regex` は、正規表現を扱います。 `compile` で作った `Regex` は、何度でも使い回せます。

REWIND

```
import std.regex as regex;

fn show(r: Result<String, StdError>) -> Unit effects {output} {
    match r {
        Ok(s) => { Out.println(s); },
        Err(e) => { Out.println(e.code); }
    }
}

fn main()->Int effects {output} {
    match regex.compile("(?P<name>[A-Za-z]+)-([0-9]+)") {
        Err(_) => { Out.println("正規表現が不正です"); },
        Ok(pattern) => {
            let input = "Alice-42 Bob-7";
            match regex.find(&pattern, &input) {
                Ok(Some(found)) => {
                    show(regex.text(&input, regex.span(&found)));       // 全体
                    match regex.group(&found, 1) {                      // 1 番目のグループ
                        Ok(Some(sp)) => { show(regex.text(&input, sp)); },
                        _ => {}
                    }
                    match regex.next(&found) {                          // 続きを探す位置
                        Some(cursor) => {
                            match regex.findFrom(&pattern, &input, cursor) {
                                Ok(Some(second)) => { show(regex.text(&input, regex.span(&secon
d))); },

                                _ => {}
                            }
                        },
                        None => {}
                    }
                },
                _ => { Out.println("見つからない"); }
            }
        }
    }
    return 0;
}
```

出力

```
Alice-42
Alice
Bob-7
```

`find` は、見つからなければ `Ok(None)` を返します。一致の範囲は、**バイト位置** の組で表され、`regex.text` で文字列に取り出します。続きを探すときは、`regex.next` が返す位置を `findFrom` に渡します。

- 対応する文法は、API の文書に従います。他の言語の正規表現と同じとは限りません。任意の後方参照や先読みを前提にしないでください。

- 正規表現と入力には、容量と処理量の予算があります（第23章）。
- 正規化した文字列で得た位置を、元の文字列で使うと、バイト列が変わっているため間違います。原文を表示する検索では、どの文字列の位置なのかを、はっきりさせてください。

第19章 JSON と CSV

設定ファイルや外部とのやりとりでは、JSON や CSV のような形式のデータを読み書きします。この章では、標準ライブラリの `std.json` と `std.csv` の使い方と、外から来るデータを扱うときの考え方を説明します。

JSON を読む

`json.parse` は、文字列を JSON の値に変換します。結果は `Result` で、JSON として正しくない文字列は `Err` になります。JSON の値は `Json` 型で、`json.get` でオブジェクトのキーを、`json.integer`、`json.text`、`json.boolean`、`json.floating`、`json.array` で値を取り出します。取り出すメソッドは、型が違うときや、キーが無いときに `None` を返します。

次の例は、プレイヤーの得点を集計します。`score` が無い項目は 0 点として数えます。

REWIND

```
import std.json as json;

fn scoreOf(item: Json) -> Int effects {} {
    match json.get(item, "score") {
        Some(s) => {
            match json.integer(s) {
                Some(n) => { return n; },
                None => { return 0; }
            }
        },
        None => { return 0; }
    }
}

fn main()->Int {
    let text = "{\"title\":\"結果\",\"players\":[{\"score\":42},{\"score\":8},{\"name\":\"x\"}]}";
    match json.parse(text) {
        Err(e) => { Out.println("JSON が不正: " + e.code); },
        Ok(root) => {
            match json.get(root, "players") {
                None => { Out.println("players がない"); },
                Some(list) => {
                    match json.array(list) {
                        None => { Out.println("players が配列ではない"); },
                        Some(items) => {
                            var total = 0;
                            for item in items { total += scoreOf(item); }
                            Out.println("${items.len()} 人、合計 ${total}");
                        }
                    }
                }
            }
        }
    }
    match json.parse("{oops") {
        Ok(_) => { Out.println("?"); },
        Err(e) => { Out.println(e.code); }
    }
    return 0;
}
```

出力

```
3 人、合計 50
Syntax
```

`match` が深く入れ子になっているのは、「ある」「型が合う」「配列である」という確認を、一つずつ明示しているからです。長くなるときは、上の `scoreOf` のように小さな関数へ分けるか、第10章の `?` が使える `Result` を返す関数にまとめてください。

> **ポイント**: JSON として正しいことと、アプリケーションが期待する形をしていることは、別の問題です。`parse` が成功したあとで、必要なキーがあるか、型が合っているかを、必ず検査してください。`json.integer` は、`Int` の値だけを返し、数字に見える文字列を自動で変換しません。

`json.stringify` は、`Json` の値を文字列に戻します。オブジェクトのキーは、並び順が決まった状態で出力されます。

大きな JSON を逐次的に読む

数十 MB の JSON を、いったん丸ごと読み込むと、その分のメモリを使います。`std.jsonStream` は、文字列を少しずつ与えながら、「オブジェクトの開始」「キー」「値」といったイベントとして順に取り出します。必要な部分だけを処理して捨てていけば、メモリの量を抑えられます。数値の token を、どの正確な型に変換するかは、利用側が別に決めます。

CSV を読む

`csv.parse` は、バイト列を表として読みます。引用符で囲まれたセルに含まれる区切りや改行も、正しく扱われます。`csv.rows` が行数、`csv.get(&表, 行, 列)` がセルを返します。範囲外は `Err` です。

REWIND

```
import std.csv as csv;

fn main()->Int {
    let source = "name,score\nAlice,42\n\"Bob, Jr.\",8\n";
    match csv.parse(Bytes(source)) {
        Err(e) => { Out.println(e.code); },
        Ok(table) => {
            var total = 0;
            for row in 1..csv.rows(&table) {          // 0 行目は見出し
                match csv.get(&table, row, 0) {
                    Ok(name) => { Out.println(name); },
                    Err(_) => {}
                }
                match csv.get(&table, row, 1) {
                    Ok(cell) => {
                        match cell.parseInt() {
                            Ok(n) => { total += n; },
                            Err(_) => {}
                        }
                    },
                    Err(_) => {}
                }
            }
            Out.println(total);
            match csv.get(&table, 9, 0) {
                Ok(_) => {},
                Err(e) => { Out.println(e.code); }
            }
        }
    }
    return 0;
}
```

出力

```
Alice
Bob, Jr.
50
Range
```

カンマで単純に `split` するだけの処理は、一般的な CSV の読み込みではありません。引用符の中のカンマや改行、`""` による引用符のエスケープに対応できないからです。CSV は必ず `std.csv`（大きなデータには増分処理の `std.csvStream`）で読んでください。

正確な値を保存する

`BigInt` や `Decimal` を `Float` に変換してから JSON の数値として保存すると、精度を失うことがあります。そのような値は、それぞれの型が用意している文字列表現で、往復させてください。日時も、タイムゾーンと精度を明示して保存します。次の章で、これらの型を学びます。

`std.config`、`std.path`、`std.args` は、設定、パス、引数の処理を補助します。パスを組み立てたことが、`File` が持つ「root の外には触れない」という制限を、取り払うわけではありません。パスの検査と、ファイルアクセスの権限は、別の仕組みです。

第20章 大きな整数・十進数・日時

失敗を取り出す補助関数

この章と次の章の標準ライブラリは、ほとんどの関数が `Result` を返します。例を短くするために、`Ok` なら中身を取り出し、`Err` なら止まる補助関数を使います。

```
fn take<T,E>(value: Result<T,E>) -> T effects {} {
    match move value {
        Ok(item) => { return move item; },
        Err(_) => { panic("失敗しました"); }
    }
}
```

実際のアプリケーションでは、`Err` を `match` や `?` で、ちゃんと扱ってください。止まってよいのは、例やテストのように、失敗が「プログラムの誤り」を意味するときだけです。

BigInt: 桁数に上限のない整数

`Int` は 64 ビットで、それを超える計算は止まります。`std.bigint` の `BigInt` は、`Float` を経由せずに、正確な整数を扱えます。

REWIND

```
import std.bigint as big;

fn take<T,E>(value: Result<T,E>) -> T effects {} {
    match move value {
        Ok(item) => { return move item; },
        Err(_) => { panic("失敗しました"); }
    }
}

fn main()->Int {
    let two = take(big.fromInt(2));
    let value = take(big.pow(&two, 100));
    Out.println(take(big.format(&value, 10)));
    return 0;
}
```

出力

```
1267650600228229401496703205376
```

`BigInt` も無限のメモリを使えるわけではなく、容量と処理量の予算があります。巨大なべき乗の結果ではなく、余りだけが必要なときは、`std.modular` の剰余演算の方が適しています。

Decimal: 十進数で正確に計算する

`Float` は二進数で値を持つので、`0.1 + 0.2` が `0.3` にならないように、十進の小数を正確には表せません。金額のように十進の桁が意味を持つ計算には、`std.decimal` の `Decimal` を使います。

丸め方は、計算のたびに **明示します**。`DecimalRounding::Exact` は、丸めが要らないときだけ成功する指定です。`HalfEven` は、ちょうど真ん中の値を偶数の側へ丸めます。

REWIND

```
import std.decimal as decimal;

fn take<T,E>(value: Result<T,E>) -> T effects {} {
    match move value {
        Ok(item) => { return move item; },
        Err(_) => { panic("失敗しました"); }
    }
}

fn main()->Int {
    let a = take(decimal.parse("0.1"));
    let b = take(decimal.parse("0.2"));
    let sum = take(decimal.add(&a, &b, 28, DecimalRounding::Exact));
    Out.println(take(decimal.format(&sum)));

    let tie = take(decimal.parse("2.5"));
    let rounded = take(decimal.quantize(&tie, 0, 28, DecimalRounding::HalfEven));
    Out.println(take(decimal.format(&rounded)));
    return 0;
}
```

出力

```
0.3
2
```

金額の計算では、丸めの規則を最初に決めてから実装してください。データベースとやりとりするときの精度の違いは、`std.dbDecimal` が扱います。

日時

日時の扱いには、時点を表す `Instant` と、時間の差を表す `Duration` が出てきます。変換や計算は、純粋な関数で、`Result` を返します。現在の時刻を観測する `clock.now` は、効果 `external` と `clock` が必要です。「日時を整形する」ことと「現在の時刻を読む」ことは、効果が違います。

タイムゾーンは、必ず名前で指定します。夏時間が終わる日には、同じ現地時刻が 2 回あり、`Earlier`（早い方）、`Later`（遅い方）、`Reject`（エラーにする）のどれかを選ぶ必要があります。次の例は、2024 年 11 月 3 日の米国東部時間 1 時 30 分を、早い方と遅い方の両方で解決して、その差を調べます。

REWIND

```
import std.datetime as datetime;

fn take<T,E>(value: Result<T,E>) -> T effects {} {
    match move value {
        Ok(item) => { return move item; },
        Err(_) => { panic("失敗しました"); }
    }
}

fn main()->Int {
    let wall = datetime.Calendar(2024, 11, 3, 1, 30, 0, 0);
    let first = take(datetime.resolve(move wall, "America/New_York", datetime.Ambiguity::Earlie
r));
    let second = take(datetime.resolve(datetime.Calendar(2024, 11, 3, 1, 30, 0, 0), "America/New_Y
ork", datetime.Ambiguity::Later));
    let gap = take(datetime.difference(&second, &first));
    Out.println(take(datetime.durationSeconds(&gap)));
    Out.println(take(datetime.format(&first)));
    Out.println(take(datetime.format(&second)));
    let tokyo = take(datetime.calendar(&first, "Asia/Tokyo"));
    Out.println(tokyo.offsetSeconds);
    return 0;
}
```

出力

```
3600
2024-11-03T05:30:00Z
2024-11-03T06:30:00Z
32400
```

同じ現地時刻の「1 時 30 分」でも、2 つの `Instant` は 1 時間（3600 秒）離れています。`Instant` を日本時間に直すと、UTC との差は 9 時間（32400 秒）です。

存在しない現地時刻（夏時間の始まりで飛ばされる時刻）は、エラーになります。年月日を入れれば、どのタイムゾーンでも一意に `Instant` が決まるわけではない、という点を覚えておいてください。

IANA のタイムゾーンデータの版は、`datetime.databaseVersion` で調べられます。データベースに日時を保存するときは、保存先の精度で丸められるか、タイムゾーンの情報をどこに残すかを、あらかじめ決めてください。

第21章 数値配列と線形代数

List と数値配列

`List<Float>` は、何でも入れられる汎用のコレクションです。大量の数値を計算するには、`std.numeric` の **数値配列** を使います。数値配列は、形（shape）を持つ、型の決まった連続したメモリの配列で、行列や多次元の配列を表せます。計算が速く、メモリも少なくて済みます。

数値配列には、`Float` の配列（`…Float` の関数）と、`Int` の配列（`…Int` の関数）があります。

関数	働き
`zerosFloat(&形)`	0 で満たした配列を作る
`fromFloat(&形, &値)`	`List<Float>` の値から作る
`getFloat(&配列, &添字)`	一つの要素を読む
`withFloat(&配列, &添字, 値)`	一つの要素を変えた **新しい** 配列を返す
`valuesFloat(&配列)`	`List<Float>` に取り出す
`reshapeFloat`、`transposeFloat`、`sliceFloat`	形の変更、転置、部分の取り出し
`sum`、`mean`、`variance`、`dot`、`matmul`	集計と行列の積
`solve`、`qr`、`leastSquares`、`eigenSymmetric`	線形代数

「形」は、各次元の大きさを並べた `List<Int>` です。`[2, 2]` なら 2 行 2 列の行列、`[3]` なら長さ 3 の列です。

連立方程式を解く

次の例は、`0x + 2y = 4`、`1x + 3y = 7` を解きます。さらに、行列の要素を書き換えて、チェックポイントへ戻すと、行列も戻ることを確かめます。

REWIND

```
import std.numeric as numeric;

fn take<T,E>(value: Result<T,E>) -> T effects {} {
    match move value {
        Ok(item) => { return move item; },
        Err(_) => { panic("数値演算に失敗しました"); }
    }
}

fn main()->Int {
    let shape = [2, 2];
    let values = [0.0, 2.0, 1.0, 3.0];
    var matrix = take(numeric.fromFloat(&shape, &values));
    let rhsShape = [2];
    let rhsValues = [4.0, 7.0];
    let rhs = take(numeric.fromFloat(&rhsShape, &rhsValues));
    let answer = take(numeric.solve(&matrix, &rhs, 0.00000000000001));
    let first = [0];
    let second = [1];
    Out.println(take(numeric.getFloat(&answer, &first)));
    Out.println(take(numeric.getFloat(&answer, &second)));

    let cell = [0, 0];
    commit saved;
    matrix = take(numeric.withFloat(&matrix, &cell, 99.0));
    Out.println(take(numeric.getFloat(&matrix, &cell)));
    publish;                  // 99 は確定済みなので、revert の後も残る
    revert saved;
    Out.println(take(numeric.getFloat(&matrix, &cell)));
    return 0;
}
```

出力

```
1
2
99
0
```

この例には、覚えておきたい点が三つあります。

1. **配列は更新ではなく、置き換えで変える**: `withFloat` は元の配列を変えず、要素を変えた新しい配列を返します。`matrix = …` と代入し直します。
2. `revert` **は配列も戻す**: `saved` へ戻すと、`matrix` は 99 を書き込む前の状態に戻ります。巨大な配列でも、変えていない部分は共有されるので、保存の費用は小さく済みます。
3. `commit` **の後の変数は、**`revert` **で消える**: `cell` を `commit` の前に作ったのは、そのためです。`commit` の後で作った変数は、`revert` の後には存在しません。

`solve` の最後の引数は、特異に近い行列を拒否するための許容値です。

計算が重いとき

数値配列の計算は、ネイティブコードで実行されます。そのため、実行ステップではなく、**ネイティブ処理量** の予算（第23章）を消費します。同期的に実行される重い計算の途中では、他のタスクへ切り替わりません。GUI と重い計算を組み合わせるときは、`numericAsync` のような、協調して進む版の関数を選んでください。

数値計算には、他にも、疎行列（`std.sparse`）、高速フーリエ変換（`std.fft`）、確率分布（`std.distributions`）、自動微分（`std.autodiff`）や最適化（`std.optimize`）があります。関数の一覧と費用の契約は、libraries/README.md を参照してください。

第22章 テスト・診断・記録と再生

テストを書く

`test fn` で書いた関数は、通常の実行では呼ばれません。`rewind test` が、それらを一つずつ実行します。中では `assert` と `assert_eq` で結果を確かめます。

REWIND

```
fn double(x: Int) -> Int = x * 2;

test fn doubles_small_numbers() {
    assert_eq(double(4), 8);
}

test fn doubles_zero() {
    assert_eq(double(0), 0);
}

fn main()->Int {
    Out.println(double(21));
    return 0;
}
```

出力

```
42
```

`rewind test` は、プロジェクト（ `rewind.toml` があるディレクトリ）に対して実行します。最初に一度、`rewind update` でロックファイルを作っておく必要があります。

コマンド

```
rewind update
rewind test --root .
```

出力

```
ok doubles_small_numbers
ok doubles_zero
```

成功したテストは、`ok 名前` と表示されます。失敗すると、そこで止まり、失敗した位置と、左右の値が表示されます。

出力

```
rewind: invalid operation: test doubles_zero: invalid operation: main.rw:8:5: assert_eq failed: le
ft=0 right=1
```

`--filter 名前` を付けると、名前に一致するテストだけを実行します。テストの中では `publish` を使えません。テストごとに独立した仮想ファイルが使われるので、あるテストが書いたファイルを、別のテストから読むことはできません。

`assert_eq` が左右を比べる規則は、`==` と同じです（第3章）。`List` どうしを比べたいときは、`freeze` してから比べてください。

エラーの診断を読む

実行時のエラーは、エラーのコード、位置、呼び出しの経路、対処のヒントの順に表示されます。

エラーになる例

```
fn divide(a: Int, b: Int) -> Int effects {} {
    return a / b;
}

fn main()->Int {
    Out.println(divide(10, 0));
    return 0;
}
```

出力

```
error[DivisionByZero]: division by zero
  --> bad.rw:2:12
  at divide (bad.rw:2:12)
  at main (bad.rw:6:17)
  at <entry> (bad.rw:5:1)
  help: Check the divisor before division or remainder.
```

- `-->` の行は、エラーが起きた `ファイル:行:列` です。行と列は 1 から数えます。
- `at` の行は、内側の呼び出しから外側へ向かう経路です。最大 32 件までで、それ以上は省略されます。変数や引数の値は表示されません。
- `help` は、対処のヒントです。

主なエラーのコードを挙げます。

コード	主な原因
`DivisionByZero`	整数を 0 で割った
`IntegerOverflow`	`Int` の範囲を超えた
`IndexOutOfBounds`	範囲外の添字
`AssertionFailed`、`Panic`	`assert`、`assert_eq`、`panic`
`UseAfterMove`	`move` した後の値を使った
`MissingEffect`	効果の宣言または許可が足りない
`InvalidContinuation`	`revert` が戻れない保存点を指した

コード	主な原因
`ExternalBoundary`	外部領域で禁止されている操作をした
`ExecutionBudgetExceeded` ほか	実行予算の超過（第23章）

プログラムが終了したときの **終了コード** も、原因を知る手がかりです。

終了コード	意味
0	正常終了
1〜63	`main` が返した値
64	コマンドの使い方の誤りなど
65	型・所有権の検査エラー、アサーションの失敗、実行時の失敗
70	`panic` や効果の不足など、原因を伴う失敗
71	予算の超過
72	外部の状態の衝突
73	公開が途中まで適用された
74	タスクの取り消し

ツールから結果を読み取りたいときは、任意のコマンドに `--diagnostic-format json` を付けます。診断が JSON として標準エラーに出力され、標準出力にはプログラムの出力だけが残ります。`diagnostic` には `code`、`message`、`line`、`column`、`frames`、`hints` などが入っています。

実行を記録して、再生する

`--record` を付けて実行すると、外から得た値（標準入力、時刻、乱数、ファイルの読み取り、外部領域の結果）が記録されます。`rewind replay` は、その記録を使って同じ実行を再現し、結果が記録と一致するかを確かめます。再生では、実際の入力や時計は使いません。

REWIND

```
fn main()->Int effects {input, output} {
    match In.readLine() {
        Some(name) => { Out.println("こんにちは、" + name + "さん"); },
        None => { Out.println("名前がありません"); }
    }
    return 0;
}
```

出力

```
こんにちは、REWINDさん
```

コマンド

```
printf 'REWIND\n' | rewind run main.rw --record trace.json
rewind replay trace.json --root .
```

一度目は標準入力から読み、二度目は記録から入力を取り出します。二度目には標準入力を渡していませんが、同じ出力になります。

この仕組みは、「再現しにくい不具合を、一度捕まえたら何度でも再現する」ために役立ちます。本番で失敗した実行を記録しておけば、手元で同じ失敗を再生して、原因を調べられます。

記録には二つの方式があります。

方式	内容
`--record-mode debug`（既定）	命令ごとの状態の索引を持つ。デバッガで巻き戻しながらステップ実行できる。大きく、長い実行には向かない
`--record-mode compact`	命令数、実行順序のダイジェスト、観測、最終状態だけ。長い実行向き

秘密の値（`Secret`、`--secret-env`、`In.readSecretLine()`）は記録に残りません。記録の形式は、コンパイラの版をまたいで互換ではありません。版を上げたら、記録し直してください。

仕事量を調べる

`rewind profile` は、関数ごと、行ごとの命令数とネイティブ処理量、実行されなかった行（カバレッジ）を表示します。測るのは時間ではなく、第23章の予算で数える **論理的な仕事量** です。既定の出力は JSON で、`--profile-format text` で読みやすい形になります。

REWIND

```
fn fib(n: Int) -> Int effects {} {
    if n < 2 { return n; }
    return fib(n - 1) + fib(n - 2);
}

fn main()->Int {
    Out.println(fib(15));
    return 0;
}
```

出力

```
610
```

コマンド

```
rewind profile main.rw --profile-format text
```

出力

```
610
task 0 main: 6 instructions exclusive, 22691 inclusive, 10 native work
task 0 fib: 22685 instructions exclusive, 22685 inclusive, 0 native work
task 0 <entry>: 11 instructions exclusive, 11 inclusive, 11 native work
coverage main.rw: 5/5 lines (uncovered: [])
```

`exclusive` は、その関数が直接実行した命令の数、`inclusive` は、呼び出した先も含めた数です。再帰した関数の `inclusive` は、他の関数の値と足し合わせてはいけません。この例では、`fib` が命令のほとんどを使っていると、一目で分かります。

エディタとデバッガ

道具	使い方
LSP	`rewind lsp --root DIR` で起動する。診断、補完、ホバー、定義・参照、名前の変更、整形に対応
デバッガ（DAP）	`--record` で記録した実行を、逆実行を含めてステップ実行できる。debugger.md
整形	`rewind fmt main.rw` 。`--check` を付けると差分の有無だけ調べる
互換性の確認	`rewind compat` は、コンパイラ、言語、成果物、記録などの版の関係を表示する

第23章 実行予算と性能

予算がある理由

REWIND のプログラムには、実行の量に上限があります。巻き戻しや再生のために、実行の履歴を保持する言語では、無限ループや暴走した処理が、環境のメモリやディスクを使い切ってしまうと、後始末が困難になるからです。上限を超えると、実行は止まり、どの予算が尽きたかが診断に出ます。

予算の種類

予算	指定	既定値	尽きたときの診断
実行ステップ（全体の累計）	`--steps N`	1,000,000	`ExecutionBudgetExceeded`
ネイティブ処理量（全体の累計）	`--native-work N`	1,000,000	`NativeWorkBudgetExceeded`
1 タスクあたりのステップ	`--task-steps N`	100,000	`TaskBudgetExceeded`
履歴メモリ	`--history-memory SIZE`	512 MiB	`HistoryMemory`
履歴ストレージ	`--history-storage SIZE`	8 GiB	`HistoryStorage`
履歴の退避しきい値	`--spill-threshold SIZE`	8 MiB	（超えた分をストレージへ退避する）

- **ステップ** は、プログラムが実行した命令の数です。
- **ネイティブ処理量** は、標準ライブラリがネイティブコードで行う処理（配列の演算、文字列の処理、`freeze` のコピーなど）の見積もりです。API ごとに、費用が文書に書かれています。
- **履歴** は、チェックポイントが保持しているデータです。

これらは **論理的な仕事量** の上限で、CPU 時間やメモリ使用量そのものではありません。どのマシンで動かしても、同じプログラムは同じ量を消費します。

既定の予算は小さい

既定の実行ステップは 100 万です。これは、思ったよりずっと小さい値です。たとえば、10 万回の足し算を繰り返すだけで、予算を使い切ってしまいます。

エラーになる例

```
fn main()->Int {
    var total = 0;
    for i in 0..100000 { total += i; }
    Out.println(total);
    return 0;
}
```

1 回の反復で、約 10 個の命令が実行されるからです。意図した計算なら、実行するときに予算を引き上げます。

コマンド

```
rewind run main.rw --steps 10000000
```

同じプログラムを、予算を引き上げて実行すれば、最後まで動きます。

REWIND

```
fn main()->Int {
    var total = 0;
    for i in 0..100000 { total += i; }
    Out.println(total);
    return 0;
}
```

出力

```
4999950000
```

エラーのヒントにある通り、予算を引き上げるのは、**仕事の量が意図したものである** と確かめた後にしてください。止まらないループを、予算を引き上げてごまかしても、解決にはなりません。

予算は巻き戻らない

予算は **累計** です。`revert` や `resume` で状態を戻しても、使った分は戻りません。そのため、巻き戻しを繰り返すプログラムも、いつかは止まります。第11章の「終わらない `resume` 」が、止まるのも同じ理由です。

ソースの中で指定する

`runtime { 設定 = 値; }` をソースに書くと、プログラムの中から予算を指定できます。設定名は `executionSteps` 、`historyMemory` 、`historyStorage` 、`spillThreshold` で、サイズには `KB` 、`MB` 、`GB` （ `KiB` 、`MiB` 、`GiB` ）を付けられます。

エラーになる例

```
runtime { executionSteps = 1000; }

fn main()->Int {
    var i = 0;
    while i < 100000 { i += 1; }
    return 0;
}
```

すでに使った分を、ソースの設定や `revert` で回復することはできません。

呼び出しの深さと、そのほかの制限

関数の呼び出しの深さにも上限があり、再帰が約 1,000 段を超えると、`ExecutionBudgetExceeded`（call depth）で止まります。

項目	上限
ソースファイル	1 MiB / module、module は 256 個まで
構文の入れ子	64 段
`freeze` の深さ	64 段（循環や資源を含む値は不可）

Linux では、コンパイラと実行時に、アドレス空間の上限（既定 2 GiB）が掛かり、`--memory-mib` で変えられます。Windows には、この OS の上限がないため、`--memory-mib` を指定するとエラーになります。論理的な予算は、どちらの OS でも有効です。OS がプロセスを強制終了したときは、`Result` として回復することはできません。

重いと感じたら

1. `rewind profile --profile-format text` で、どの関数・どの行に仕事が集中しているかを調べます（第22章）。
2. ループで数値を一つずつ処理しているなら、数値配列（第21章）で、まとめて処理できないか考えます。ネイティブコードで実行されるため、同じ仕事を、ずっと少ないステップで処理できます。
3. 履歴のメモリが多いなら、不要になったチェックポイントを `drop` で捨てます（第11章）。
4. 実際のプログラムは、ステップ数やネイティブ処理量が、他の API の費用に比例することがあります。各 API の費用は、libraries/README.md と標準ライブラリの文書に書かれています。

処理が重くなる原因は、プログラムの書き方だけでなく、データの量や、チェックポイントに保持しているデータの量にもあります。「論理的な仕事量」を測るという考え方で、まず測り、それから直す、という順番で取り組んでください。

第24章 外の世界とつなぐ: データベース・通信・GUI

この章では、プログラムの外にあるデータベース、ネットワーク、画面とやりとりする方法を、一つずつ見ていきます。どれも、「巻き戻せない作用」を含みます。第12章の「戻せるものと戻せないものの境目」を、もう一度思い出しながら読んでください。

共通の約束

通信やデータベースの操作は、REWIND の外の相手とやりとりします。そのため、次の三つの約束に従います。

1. **開始は `external { }` の中、結果の待機は外。**領域の中では、操作を「依頼する」だけで、`await` はできません。依頼すると `Task` が返るので、領域を出てから `await` します。
2. **結果は二重の `Result`。**`Task<Result<T,E>>` を `await` すると、`Result<Result<T,E>,TaskError>` になります。外側は「タスクとして待てたか」、内側は「その API としての成否」です。
3. **外へ及んだ作用は、`revert` で消えない。**送信した通信も、データベースに書き込んだ行も、巻き戻せません。

どの操作にも、対応する効果（`network`、`db`、`gui`）と、`external`、`tasks` の許可が必要です。許可するときは、実行時に `--allow-effects external,db,tasks` のように指定します。

データベース

`std.db` は、SQLite と PostgreSQL を扱います。SQLite の `":memory:"` は、そのプログラムの中だけにある、独立したデータベースを作ります。ファイル名を指定すると、プログラムが終わった後もデータが残ります。

次の例は、表を作り、値を挿入し、問い合わせで読み出します。

REWIND

```
import std.db as db;

fn take<T,E>(value: Result<T,E>) -> T effects {} {
    match move value {
        Ok(item) => { return move item; },
        Err(_) => { panic("データベース操作に失敗しました"); }
    }
}

fn main()->Int effects {external, db, tasks, output} {
    // 1. 接続する。依頼は external の中、待機は外
    var opening: Option<Task<Result<DbConnection, DbError>>> = None;
    external { opening = Some(db.sqlite(":memory:", false, 5000)); }
    match opening {
        None => { panic("接続の Task がない"); },
        Some(task) => {
            let connection = take(take(await task));

            // 2. 表を作る
            var writing: Option<Task<Result<Int, DbError>>> = None;
            external { writing = Some(db.execute(&mut connection, "CREATE TABLE scores (name TEXT,
score INTEGER)", freeze(List<DbValue>()), 5000)); }
            match writing { Some(t) => { take(take(await t)); }, None => { panic("なし"); } }

            // 3. 値をパラメータとして渡して挿入する
            let values = List<DbValue>();
            values.push(DbValue::Text("Alice"));
            values.push(DbValue::Int(42));
            external { writing = Some(db.execute(&mut connection, "INSERT INTO scores VALUES (?1,
?2)", freeze(move values), 5000)); }
            match writing { Some(t) => { Out.println(take(take(await t))); }, None => { panic("な
し"); } }

            // 4. 問い合わせて、結果を読む
            var querying: Option<Task<Result<DbCursor, DbError>>> = None;
            external { querying = Some(db.query(&mut connection, "SELECT name, score FROM scores",
freeze(List<DbValue>()), 5000)); }
            match querying {
                None => { panic("なし"); },
                Some(t) => {
                    let cursor = take(take(await t));
                    var reading: Option<Task<Result<DbBatch, DbError>>> = None;
                    external { reading = Some(db.next(&mut cursor, 64, 1048576, 5000)); }
                    match reading {
                        None => { panic("なし"); },
                        Some(rt) => {
                            let batch = take(take(await rt));
                            Out.println(take(db.text(batch.rows.get(0), 0)));
                            Out.println(take(db.integer(batch.rows.get(0), 1)));
                        }
                    }
                }
            }
```

```
        }
    }
    // 5. 後始末の完了を待つ
    var cleanup: Option<Task<Result<Unit, DbError>>> = None;
    external { cleanup = Some(db.waitForCleanup(5000)); }
    match cleanup { Some(t) => { take(take(await t)); }, None => { panic("なし"); } }
    return 0;
}
```

出力

```
1
Alice
42
```

長く見えますが、手順は 5 つだけです。接続、表の作成、挿入、問い合わせ、後始末で、どの手順も「`external` の中で依頼して、外で `await` する」形をしています。`take(take(await t))` が二重になっているのは、先ほどの二重の `Result` のためです。

`DbConnection` や `DbCursor` は、持ち主が一人の資源です。スコープを出るときに解放が始まりますが、`waitForCleanup` で完了を待つと、後始末の失敗も確実に受け取れます。

値は、パラメータで渡す

SQL に値を埋め込むとき、文字列を連結してはいけません。上の例の `?1`、`?2` のように、プレースホルダを使い、`DbValue` の列を別に渡します。文字列連結は、SQL インジェクションの原因になります。値で置き換えられない部分（表名や列名の動的な指定など）は、別に検査してください。

`NULL` は `DbValue::Null` で、整数の `0` や空文字列とは別物です。`db.integer` は整数のセル、`db.text` は文字列のセルを読みます。型が違うと、暗黙には変換されず、`Err` になります。

VM のコミットとデータベースのコミットは別

操作	対象
`commit 名前`	REWIND の計算状態の保存
`revert 名前`	保存した計算状態への復元
`publish`	保留中のファイルや出力の確定
データベースの `begin` / `commit` / `rollback`	データベース側のトランザクション

`revert` は、データベースの `rollback` ではありません。データベースへ保存された行を、REWIND の `revert` で消すことはできません。複数の SQL をまとめて、成功か失敗のどちらかにしたいときは、データベースのトランザクションを使います（`std.dbTransaction`）。

PostgreSQL では、パスワードを含む接続文字列を `Secret` として渡します。TLS の証明書とホスト名は検証されます。自動の再接続や、書き込みの再試行は行われません。送信した後で結果が分からなくなったときは、データベースの実際の状態を確かめてから判断してください。

HTTP の通信

`std.http` の `get` は、`Task<Result<HttpResponse,HttpError>>` を返します。次の例は、URL を引数で受け取り、応答の状態コードと本文を表示します。

REWIND

```
import std.http as http;
import std.bytes as bytes;

fn main()->Int effects {args, external, network, tasks, output} {
    let arguments = Args.all();
    if arguments.len() != 1 {
        Err.println("使い方: rewind run main.rw --allow-effects external,network,tasks -- URL");
        return 1;
    }
    var pending: Option<Task<Result<HttpResponse, HttpError>>> = None;
    external { pending = Some(http.get(arguments.get(0), 10000, 65536)); }
    match pending {
        None => { panic("Task がない"); },
        Some(task) => {
            match await task {
                Err(_) => { Err.println("取り消されました"); },
                Ok(response) => {
                    match response {
                        Err(e) => { Err.println(e.code + " (" + e.phase + ")"); },
                        Ok(r) => {
                            Out.println(r.status);
                            match bytes.decode(r.body) {
                                Ok(text) => { Out.println(text); },
                                Err(e) => { Err.println(e.code); }
                            }
                        }
                    }
                }
            }
        }
    }
    return 0;
}
```

出力

```
200
hello from server
```

この例は、手元で簡単なサーバーを立てて実行して確かめました。実際に使うときは、`rewind run main.rw --allow-effects args,external,network,tasks -- https://example.com/` のように、URL を渡します。

通信を扱うときの注意を挙げます。

- `get` の引数は、URL、期限（ミリ秒）、応答本文の最大バイト数です。期限は実時間で、第15章のタスクの `timeout`（論理的なステップ数）とは別の単位です。
- リダイレクトや再試行は、自動では行われません。
- 状態コードが `404` のような応答と、接続や TLS の失敗は、別々に扱います。`200` が返ったことは、本文が期待した形式であることを意味しません。
- HTTPS では、証明書の連鎖とホスト名が検証されます。診断を避けるために、検証を無効にする設計にはしないでください。
- 送信した後に時間切れになったときは、相手が処理したかどうかは分かりません。送信は `revert` で取り消せないので、再試行するときは、相手側の重複防止（冪等性のキーなど）も考えてください。
- 記録付きの実行では、通信の結果が記録され、再生では再送されません。再生に成功したことは、実際のサーバーに再び送れたという意味ではありません。

大きな本文を少しずつ受け取る `download`、`read` や、生の TCP を扱う `std.tcp`、HTTP サーバーを作る `std.httpServer`、`std.httpRouter` もあります。TCP の `read` は、メッセージの境界を保証しません。プログラムの側で、長さや区切りを決めて処理してください。

ネイティブ GUI

`std.gui` は、ネイティブのウィンドウを作ります。GUI の状態を、三つの層に分けて考えると整理できます。

層	内容
モデル	数値や入力内容など、アプリケーションのデータ
View	部品とその配置
OS の画面	`present` して `publish` した画面

モデルや View を変えても、`present` して `publish` するまで、画面は変わりません。

次の例は、ラベルとボタンを表示し、閉じる操作を待ちます。実行には `gui` の許可と、画面を表示できる環境が必要です。

REWIND

```
import std.gui as gui;

fn checked(value: Result<Unit, gui.GuiError>) -> Unit effects {} {
    match value {
        Ok(_) => {},
        Err(e) => { panic(e.code); }
    }
}

fn main()->Int effects {gui, output} {
    match gui.window("Hello GUI", 320, 180) {
        Err(e) => { panic(e.code); },
        Ok(view) => {
            checked(gui.label(&mut view, "message", "Hello, REWIND", 20, 20, 280, 40));
            checked(gui.button(&mut view, "close", "Close", 20, 90, 120, 40));
            checked(gui.present(&view));
            publish;
            var running = true;
            while running {
                match gui.nextEvent() {
                    Err(e) => { panic(e.code); },
                    Ok(event) => {
                        match gui.dispatch(&mut view, event) {
                            Some(gui.Action::Close) => { running = false; },
                            Some(gui.Action::Activate(id)) => { if id == "close" { running = fals
e; } },
                            _ => {}
                        }
                    }
                }
            }
            checked(gui.close());
        }
    }
    return 0;
}
```

`gui.window` が View を作り、`gui.present` と `publish` で画面に表示します。`gui.nextEvent` は入力を待ち、`gui.dispatch` は座標やキーを、部品に対応する動作（`Action`）に変換します。

Undo を作る

GUI は、巻き戻しの力がよく見える場面です。次の例は、`Add` で数を増やし、`Undo all` で最初の状態に戻します。

REWIND

```
import std.gui as gui;

fn checked(value: Result<Unit, gui.GuiError>) -> Unit effects {} {
    match value {
        Ok(_) => {},
        Err(e) => { panic(e.code); }
    }
}

fn main()->Int effects {gui, output} {
    match gui.window("カウンター", 360, 180) {
        Err(e) => { panic(e.code); },
        Ok(view) => {
            var count = 0;
            checked(gui.label(&mut view, "count", "0", 20, 20, 300, 40));
            checked(gui.button(&mut view, "add", "Add", 20, 80, 140, 40));
            checked(gui.button(&mut view, "undo", "Undo all", 180, 80, 140, 40));
            var running = true;
            commit ready;                        // 最初の状態を保存する
            checked(gui.present(&view));
            publish;
            while running {
                match gui.nextEvent() {
                    Err(e) => { panic(e.code); },
                    Ok(event) => {
                        match gui.dispatch(&mut view, event) {
                            Some(gui.Action::Activate(id)) => {
                                if id == "add" {
                                    count += 1;
                                } else if id == "undo" {
                                    revert ready;             // 状態を戻す
                                    gui.continueInput();      // すでに届いた入力は読み飛ばす
                                }
                                checked(gui.setText(&mut view, "count", count.format()));
                            },
                            Some(gui.Action::Close) => { running = false; },
                            _ => {}
                        }
                    }
                }
                if running {
                    checked(gui.present(&view));
                    publish;
                }
            }
            checked(gui.close());
            Out.println(count);
        }
    }
    return 0;
}
```

出力

```
1
```

この例の入力は、「Add を 2 回、Undo、Add、閉じる」です。`Undo all` で `count` が 0 に戻り、その後の `Add` で 1 になるので、最後に `1` が表示されます。

ポイントは二つです。

- `revert ready;` が戻すのは、`count` などの計算状態です。**画面は戻りません。**`present` と `publish` で、戻った状態を画面へ反映します。
- `gui.continueInput()` は、すでに届けた入力を読み飛ばします。これが無いと、巻き戻した入力の位置から、古いクリックをもう一度読んでしまい、Undo の直後に古い `Add` が効いてしまいます。

GUI の制限と環境

- 一画面の幅と高さは、それぞれ 64〜4096 ピクセルです。部品は最大 2048 個で、id は 128 バイト以内の空でない文字列です。
- 複数のウィンドウ（`std.guiWindows`）は、最大 16 枚です。
- Windows では Win32/GDI、Linux では X11 を使います。Linux では、`DISPLAY` と、X11 のライブラリ、フォントが必要です。
- 日本語入力には、OS 側の IME が必要です。変換中の未確定文字列は、OS 側の状態で、確定した文字だけがプログラムの入力になります。

入力は記録できます。`--gui-events EVENTS.json` を付けると、実際のウィンドウを開かず、ファイルのイベント列を与えて実行できます。この章の GUI の例も、この方法で確かめています。モデルの更新や終了の回帰テストに便利ですが、ネイティブの IME や描画の確認にはなりません。

フォーム（`std.guiForm`）、表（`std.guiTable`）、メニュー（`std.guiMenu`）、ファイル選択ダイアログ（`std.guiDialog`）といった部品も用意されています。通信やデータベースの処理を、GUI のイベントループと組み合わせるときは、チャネル（第15章）で結果を通知する構成にすると、画面と外部作用の境界を保てます。

第25章 実習: 在庫を「全部反映するか、一つも反映しないか」で更新する

最後に、これまでの章の内容を一つのプログラムにまとめます。題材は、在庫の更新です。標準入力から、1 行に 1 つずつ命令を読みます。

```
add apple 5
remove apple 3
```

命令を一つずつ反映していき、**途中で一つでも失敗したら、それまでの反映もすべて取り消す**、という動作にします。データが半端に更新された状態を、残さないためです。

設計を考える

最初に、部品を分けて考えましょう。

部品	役割	使う機能
`Command`	命令を表す型	`enum`（第7章）
`parse`	1 行を `Command` に変換する	`Result` と `?`（第10章）
`apply`	在庫に命令を反映する	`&mut` の借用（第9章）
`have`	在庫の数を調べる	`Map.get` と `Option`（第6章、第10章）
`main`	読み込み、取り消し、報告	`commit` と `revert`（第11章）、`publish`（第12章）

計算の部分（`parse`、`apply`、`have`）は、外部とやりとりしないので、`effects {}` と書けます。入出力は `main` と `report` だけに集めます。こうして分けておけば、計算の部分を、入出力なしでテストできます（第22章）。

プログラム

REWIND

```
enum Command {
    Add(String, Int),
    Remove(String, Int)
}

fn parse(line: String) -> Result<Command, String> effects {} {
    let parts = line.split(" ");
    if parts.len() != 3 {
        return Err("形式が違います: " + line);
    }
    let amount = parts.get(2).parseInt()?;
    let item = parts.get(1);
    if parts.get(0) == "add" { return Ok(Command::Add(item, amount)); }
    if parts.get(0) == "remove" { return Ok(Command::Remove(item, amount)); }
    return Err("未知の命令です: " + parts.get(0));
}

fn have(stock: &Map<String, Int>, item: String) -> Int effects {} {
    match stock.get(item) {
        Some(n) => { return n; },
        None => { return 0; }
    }
}

fn apply(stock: &mut Map<String, Int>, command: Command) -> Result<Unit, String> effects {} {
    match command {
        Command::Add(item, n) => {
            stock.set(item, have(stock, item) + n);
            return Ok(());
        },
        Command::Remove(item, n) => {
            let current = have(stock, item);
            if current < n {
                return Err(item + " の在庫が足りません");
            }
            stock.set(item, current - n);
            return Ok(());
        }
    }
}

fn report(stock: &Map<String, Int>) -> Unit effects {output} {
    for item in stock.keys() {
        Out.println("${item}: ${have(stock, item)}");
    }
}

fn main()->Int effects {input, output} {
    let stock = Map<String, Int>();
    stock.set("apple", 1);
```

```
    var count = 0;
    commit start;                                   // 反映前の状態を保存する
    while true {
        match In.readLine() {
            None => { break; },
            Some(line) => {
                let result = match parse(line) {
                    Ok(command) => { apply(&mut stock, command) },
                    Err(message) => { Err(message) }
                };
                match result {
                    Ok(_) => { count += 1; },
                    Err(message) => {
                        Out.println("失敗: " + message);
                        publish;                        // 理由は取り消されないよう、先に確定する
                        revert start;                   // 在庫への変更をすべてなかったことにする
                        Out.println("1 件も反映せず、在庫は元のままです");
                        report(&stock);
                        return 1;
                    }
                }
            }
        }
    }
    Out.println("${count} 件を反映しました");
    report(&stock);
    return 0;
}
```

出力

```
失敗: apple の在庫が足りません
1 件も反映せず、在庫は元のままです
apple: 1
```

標準入力に、`add apple 5`、`remove apple 9`、`add pear 2` の 3 行を渡した結果です。2 行目で在庫が足りなくなり、1 行目の `add apple 5` も取り消され、`apple` は最初の 1 のままです。`return 1` で終了したので、終了コードは 1 です。

すべての命令が成功すると、次のようになります。

REWIND

```
enum Command {
    Add(String, Int),
    Remove(String, Int)
}

fn parse(line: String) -> Result<Command, String> effects {} {
    let parts = line.split(" ");
    if parts.len() != 3 {
        return Err("形式が違います: " + line);
    }
    let amount = parts.get(2).parseInt()?;
    let item = parts.get(1);
    if parts.get(0) == "add" { return Ok(Command::Add(item, amount)); }
    if parts.get(0) == "remove" { return Ok(Command::Remove(item, amount)); }
    return Err("未知の命令です: " + parts.get(0));
}

fn have(stock: &Map<String, Int>, item: String) -> Int effects {} {
    match stock.get(item) {
        Some(n) => { return n; },
        None => { return 0; }
    }
}

fn apply(stock: &mut Map<String, Int>, command: Command) -> Result<Unit, String> effects {} {
    match command {
        Command::Add(item, n) => {
            stock.set(item, have(stock, item) + n);
            return Ok(());
        },
        Command::Remove(item, n) => {
            let current = have(stock, item);
            if current < n {
                return Err(item + " の在庫が足りません");
            }
            stock.set(item, current - n);
            return Ok(());
        }
    }
}

fn report(stock: &Map<String, Int>) -> Unit effects {output} {
    for item in stock.keys() {
        Out.println("${item}: ${have(stock, item)}");
    }
}

fn main()->Int effects {input, output} {
    let stock = Map<String, Int>();
    stock.set("apple", 1);
    var count = 0;
    commit start;
    while true {
```

```
        match In.readLine() {
            None => { break; },
            Some(line) => {
                let result = match parse(line) {
                    Ok(command) => { apply(&mut stock, command) },
                    Err(message) => { Err(message) }
                };
                match result {
                    Ok(_) => { count += 1; },
                    Err(message) => {
                        Out.println("失敗: " + message);
                        publish;
                        revert start;
                        Out.println("1 件も反映せず、在庫は元のままです");
                        report(&stock);
                        return 1;
                    }
                }
            }
        }
    }
    Out.println("${count} 件を反映しました");
    report(&stock);
    return 0;
}
```

出力

```
3 件を反映しました
apple: 3
pear: 2
```

読み解きのポイント

`revert` **の前に** `publish` **している理由。** `revert start;` は、`start` より後に作られた変数を消します。`message` もその一つで、`revert` の後では使えません。そこで、失敗の理由を、`revert` の前に出力して `publish` で確定しています。確定済みの出力は、`revert` で取り消されません。この「戻る前に、残したい情報を外へ出す」という順序は、チェックポイントを使うときの基本です。

`count` **を** `commit` **の前に宣言している理由。** `commit` の後で作った変数は、`revert` で消えます。ループの中で `revert` するので、ループの外にある変数は、`commit` より前に作っておく必要があります。

`return 1` **でも出力は公開される。** `main` が戻れば、保留中の出力は、戻り値にかかわらず公開されます。「反映せず、在庫は元のまま」の出力と、`report` の出力は、この自動公開で外に出ます。

借用した `Map` **を** `for` **で回す。** `report` は `&Map` を受け取りますが、`for item in stock.keys()` と、そのままキーを回せます。`keys()` が返すのはキーの列を進める反復子で、`Map` そのものは変更しません。回している間に `stock.set(...)` のような変更を書くと、借用中の変更として拒否されます。

発展: 取り組んでみよう

1. `remove` で在庫が 0 になった品目を、`Map` から消すように変えてみましょう（ `stock.remove(item)` ）。
2. `parse` と `apply` に対する `test fn` を書き、`rewind test` で確かめましょう。入出力を持たない関数なので、標準入力を用意する必要はありません。
3. 実行を `--record` で記録して、`rewind replay` で再生してみましょう。標準入力を渡さなくても、同じ結果が再現されるはずです。
4. 命令が千件を超えると、既定の実行ステップ（第23章）を使い切るかもしれません。`rewind profile` で、どこに仕事が集中しているか調べてみましょう。

付録A 標準ライブラリの地図

標準ライブラリの module は、`import std.名前 as 名前;` で使います。各関数の型、費用、失敗条件は、SDK の `share/rewind/doc/std/` と libraries/README.md にあります。ここでは、どんな module があるかの地図だけを示します。

分類	module
値と変換	option、result、error、failures、issues、number、integer、modular、bigint、decimal、math、bits、bytes、units、layout
文字列とテキスト	text、unicode、textIndex、textEdit、regex、stringSearch、url、path
日時	datetime、clock
コレクションとアルゴリズム	collections、map、orderedMap、sequence、sort、search、heap、deque、bitset、disjointSet、rollbackSet、fenwick、segment、lazySegment、range、trie、suffix、dp、table
グラフと幾何	graph、graphLarge、graphLargeAsync、flow、matching、geometry
データ形式と解析	json、jsonStream、csv、csvStream、config、args、scanner、stream、parser、schema、logfmt、compress、image
履歴と状態	snapshot、history、historyGraph、timeline、changes、persistentSet、memo、lazy、lens
数値計算	numeric とその Async 系、matrix、sparse、sparseAsync、fft、fftAsync、distributions
機械学習	autodiff 系、optimize、training、models、evaluation、plot
タスクと通信	task、taskError、retry、external、http、httpHeaders、httpRouter、httpServer、tcp
データベース	db、dbDatetime、dbDecimal、dbMigration、dbTransaction
GUI	gui、guiDialog、guiField、guiForm、guiMenu、guiTable、guiWindows
テスト支援	proptest

「履歴と状態」の module は、REWIND ならではの道具です。たとえば、`std.timeline` は Undo と Redo の履歴を、`std.history` は名前付きのスナップショットの登録簿を、`std.snapshot` は一部のデータの独立した複製を扱います。チェックポイントは、名前を実行前に決めておく必要があるので、実行中に増えていく履歴には、これらの module を使います。

付録B よくあるエラーと対処

実行前に出るエラーと、実行中に止まるエラーのうち、とくに出会いやすいものを挙げます。

表示	原因	対処
`cannot assign to let binding`	`let` の名前に代入した	`var` にする。あるいは新しい名前を使う
`duplicate binding`	同じスコープで同じ名前を二度作った	名前を変えるか、内側のスコープに移す
`function returning Int needs a return on every path`	値を返す関数に、`return` を通らない経路がある	すべての分岐で `return` する
`non-exhaustive match`	`match` で扱っていない場合がある	抜けている variant を足すか、`_` で受ける
`match arm expects`	分岐の値の型がそろっていない	すべての分岐を同じ型にする
`passing a mutable owner requires move or freeze`	変更できる値を、`move` なしで渡した	`move` する、`&` や `&mut` で貸す、`freeze` する
`UseAfterMove`	`move` した値をまた使った	渡した後は使わない。必要なら先に `freeze` する
`cannot mutate through a shared borrow`	`&` で借りた値を変更しようとした	引数の型を `&mut` にする
`conflicting borrows`	同じ値を二つの `&mut` で貸そうとした	貸す期間を分ける
`mutable owner alias`	`List` や `struct` を、代入で別の名前に持たせようとした	`move`、借用、`freeze` のいずれかを選ぶ
`MissingEffect`	効果の宣言か、実行時の許可が足りない	`effects { … }` に足す。実行時は `--allow-effects` で許可する
`IntegerOverflow`	`Int` の範囲（64 ビット）を超えた	`BigInt` を使うか、範囲を確かめる
`DivisionByZero`	0 で割った	割る前に確かめる
`IndexOutOfBounds`	添字が範囲の外	`len()` と比べてから使う
`InvalidContinuation`	`revert` が戻れない保存点を指した	同じ関数の、有効なスコープの保存点に戻す
`ExternalBoundary`	`external` の中で禁止された操作をした	`await` や `publish` などを、領域の外へ出す
`ExecutionBudgetExceeded`	実行ステップの予算を使い切った	無限ループでないか確かめ、意図した仕事なら `--steps` を引き上げる

表示	原因	対処
`unknown name`	存在しない、または公開されていない名前を使った	綴りと `pub` を確かめる

付録C 用語集

用語	意味
束縛	名前と値の結び付き。`let` と `var` で作る
所有者	変更できる値（`List`、`Map`、`struct` など）を持っている束縛
共有値	自由にコピーして共有できる値（`Int`、`String`、`record` など）
借用	所有権を渡さずに、一時的に値を貸すこと。`&`（読むだけ）、`&mut`（変更する）
チェックポイント	計算状態に付ける名前付きの保存点
保留中（pending）	まだ外へ確定していない出力や書き込み
公開（publish）	保留中の入出力を外へ確定すること。確定したものは戻せない
効果（effect）	関数が外に対して行う操作の種類。`effects { … }` で宣言する
外部領域	外の世界から得る値や操作を閉じ込め、結果を記録する `external { }`
予算	実行ステップなど、実行の量の上限
記録と再生	外から得た値を記録し、同じ実行を再現すること
診断	エラーの表示。コード、位置、呼び出しの経路、対処のヒントを持つ

付録D 現状の制約

この本の版（REWIND 3.6.0）の実装で、期待と異なる動作をする点を挙げます。今後の版で変わる可能性があります。

- `?` **のエラー型の不一致は、実行時に見つかります。** `Err` の経路を通ったときの実行時エラーで、実行前には拒否されません（第10章）。
- **ジェネリクスの制約に、存在しない名前を書いても拒否されません。** `T: Bogus` のように書いても、エラーになりません。
- `Out.println` **が** `struct` **や** `record`**、**`Json` **などを表示する形式は、内部表現に近く、安定していません。**人が読む文字列は、自分で組み立ててください。
- `List` **と** `Map` **を** `==` **や** `assert_eq` **で比べると、中身ではなく同一性を調べます。**比べたいときは `freeze` します（第3章）。
- `return` **を** `branch` **の中で使うと、そこで関数から戻りますが、状態は戻りません。**状態を元に戻したいときは、ブロックを最後まで実行して終えてください（第12章）。

付録E 次に読むもの

知りたいこと	読むもの
言語の正確な定義	language-reference.md
インストールと SDK の検証	getting-started.md
標準ライブラリの各関数	libraries/README.md
ネイティブ GUI	gui.md
デバッガ	debugger.md
版ごとの変更点	`docs/REWIND_v*.md`